BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH

**TRẦN VĂN NHANH - 2251052079**

# Phát triển hệ thống hỗ trợ học từ vựng tiếng Anh sử dụng ASP.NET Core và ReactJS kết hợp gamification

ĐỒ ÁN NGÀNH
NGÀNH CÔNG NGHỆ THÔNG TIN

Giảng viên hướng dẫn:
**ThS. NGUYỄN TRUNG HẬU**

TP. Hồ Chí Minh, 2025

# Chapter 1

# Giới thiệu

## 1.1 Lý do chọn đề tài

Trong thời kỳ hội nhập quốc tế, tiếng Anh trở thành một kỹ năng thiết yếu cho học tập, công việc và giao tiếp xuyên biên giới. Trong số các năng lực ngoại ngữ, vốn từ vựng đóng vai trò nền tảng và có mối quan hệ chặt chẽ với bốn kỹ năng nghe, nói, đọc, viết; do đó đầu tư vào việc học và quản lý từ vựng có tác động trực tiếp tới khả năng sử dụng ngôn ngữ [1].

Tuy nhiên, thực tiễn học tập cho thấy nhiều người học gặp khó khăn trong việc ghi nhớ từ vựng lâu dài và duy trì thói quen học đều đặn. Các nghiên cứu tâm lý học đã chứng minh rằng kỹ thuật lặp lại có khoảng cách (spaced repetition) và thực hành truy hồi (retrieval practice) có hiệu quả rõ rệt trong việc củng cố trí nhớ dài hạn [2, 3]. Mặc dù các ứng dụng hiện nay như Anki hay Duolingo đã khai thác các nguyên lý này, nhưng vẫn tồn tại sự tách biệt giữa thuật toán ghi nhớ khô khan và cơ chế gắn kết cảm xúc lâu dài. Động lực ngoại tại (như chuỗi ngày học - streak) thường gây áp lực tâm lý thay vì tạo ra sự hứng thú nội tại thông qua sự phát triển cá nhân.

Bên cạnh đó, việc xây dựng một hệ thống học tập hiện đại đòi hỏi một nền tảng kỹ thuật vững chắc, có khả năng mở rộng và bảo trì cao. Từ những thực trạng trên, đề tài **"Xây dựng hệ thống học từ vựng tiếng Anh kết hợp gamification"** được lựa chọn nhằm mục tiêu: (1) Giải quyết bài toán ghi nhớ bằng thuật toán khoa học; (2) Duy trì động lực bền vững thông qua hệ thống thú ảo gắn liền với tiến trình học; và (3) Hiện thực hóa giải pháp trên một kiến trúc phần mềm tiên tiến (Clean Architecture) sử dụng hệ sinh thái .NET 8 và ReactJS.

## 1.2 Mục tiêu đề tài

Mục tiêu chính của đề tài là phát triển một hệ thống học từ vựng trực tuyến giúp người học ghi nhớ hiệu quả hơn và duy trì thói quen học lâu dài. Hệ thống áp dụng thuật toán SuperMemo-2 (SM-2) để tối ưu hóa lộ trình ôn tập cá nhân hóa. Đồng thời, các yếu tố Gamification chuyên sâu như hệ thống tiến hóa thú ảo (Pet System) được tích hợp để biến quá trình học thành một hành trình chinh phục, giúp thỏa mãn các nhu cầu tâm lý về năng lực và sự tự chủ của người học.

## 1.3 Phạm vi đề tài

Đề tài tập trung vào việc xây dựng một ứng dụng web (Web Application) đa nền tảng hướng tới người học tiếng Anh ở trình độ cơ bản đến trung cấp.

- **Về chức năng:** Tập trung vào quản lý bộ từ cá nhân, thực hiện phiên học với 4 dạng tương tác (Flashcard, Fill-in-blank, Multiple Choice, Listening) và hệ thống quản lý thú ảo.
- **Về kỹ thuật:** Thiết kế và hiện thực hệ thống theo kiến trúc **Onion Architecture** (Clean Architecture), đảm bảo tính tách biệt giữa logic nghiệp vụ và hạ tầng công nghệ.
- **Giới hạn:** Đề tài không đi sâu vào việc giảng dạy ngữ pháp chuyên sâu hay giao tiếp thời gian thực, mà tập trung tối ưu hóa quy trình học và quản lý từ vựng.

## 1.4 Phương pháp tiếp cận

Đề tài được thực hiện dựa trên quy trình xây dựng phần mềm chuyên nghiệp kết hợp nghiên cứu thực nghiệm:

1. **Nghiên cứu lý thuyết:** Tổng hợp tài liệu về đường cong quên lãng, thuật toán SM-2 và Thuyết Tự Quyết (SDT) để làm cơ sở thiết kế logic hệ thống.
2. **Quy trình phát triển:** Áp dụng mô hình **Agile/Scrum** rút gọn để phát triển hệ thống theo từng giai đoạn, cho phép kiểm thử và tinh chỉnh liên tục các tính năng Gamification.
3. **Thiết kế kiến trúc:** Sử dụng ngôn ngữ UML để mô hình hóa hệ thống. Backend được tổ chức theo mô hình **Onion Architecture** với các Design Patterns tiêu biểu như Repository, Unit of Work và DTO để đảm bảo tính dễ kiểm thử (Testability).
4. **Hiện thực hóa (Tech Stack):**
   - **Backend:** .NET 8, ASP.NET Core Web API, Entity Framework Core, SQL Server.
   - **Frontend:** ReactJS, TypeScript, Vite, Tailwind CSS.
5. **Kiểm thử và Đánh giá:** Thực hiện Unit Test cho logic nghiệp vụ và kiểm thử chức năng trên các kịch bản thực tế để đối chiếu với các giả thuyết nghiên cứu ban đầu.

## 1.5 Bố cục của khóa luận

Khóa luận được cấu trúc thành bốn chương chính:

**Chương 1: Giới thiệu** - Trình bày bối cảnh, lý do chọn đề tài, mục tiêu, giả thuyết và phương pháp tiếp cận.

**Chương 2: Cơ sở lý thuyết và công trình liên quan** - Phân tích các nguyên lý khoa học về trí nhớ, tâm lý học hành vi và đánh giá các ứng dụng hiện có để chỉ ra khoảng trống nghiên cứu.

**Chương 3: Phân tích, thiết kế và hiện thực hệ thống** - Đây là chương trọng tâm, mô tả chi tiết kiến trúc Onion, thiết kế UML, mô hình dữ liệu và trình bày kết quả triển khai thực tế (Demo) cùng các đánh giá kỹ thuật.

**Chương 4: Kết luận và hướng phát triển** - Tổng kết kết quả đạt được đối chiếu với mục tiêu đề ra, tự đánh giá ưu khuyết điểm và đề xuất hướng mở rộng.

# Chapter 2

# Cơ sở lý thuyết

## 2.1 Lý thuyết về học từ vựng và thuật toán lặp lại có khoảng cách

Trong lĩnh vực tiếp thu ngôn ngữ thứ hai, từ vựng không chỉ là những đơn vị ngôn ngữ rời rạc mà còn là nền tảng cốt lõi chi phối khả năng vận hành cả bốn kỹ năng nghe, nói, đọc và viết. Tuy nhiên, thách thức lớn nhất đối với người học không nằm ở việc tiếp nhận thông tin mới, mà ở khả năng duy trì thông tin đó trong trí nhớ dài hạn. Thực tế này được làm sáng tỏ bởi Hermann Ebbinghaus thông qua khái niệm "đường cong quên lãng" (forgetting curve), mô tả sự suy giảm theo hàm mũ của dấu vết trí nhớ ngay sau khi quá trình học kết thúc.

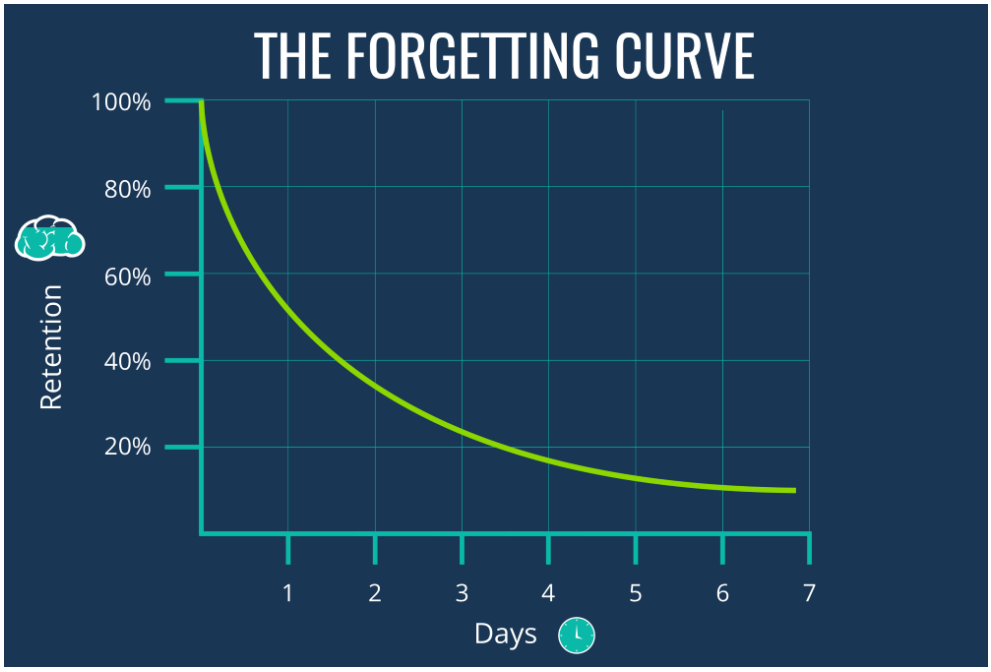


Figure 2.1: Đường cong quên lãng.

Quy luật tự nhiên này chỉ ra rằng nếu không có sự can thiệp kịp thời, phần lớn kiến thức sẽ biến mất chỉ sau một thời gian ngắn. Để đối phó với hiện tượng quên lãng, kỹ thuật lặp lại có khoảng cách (Spaced Repetition - SRS) đã ra đời như một giải pháp dựa trên bằng chứng khoa học. Thay vì nhồi nhét thông tin trong một thời gian ngắn (massed practice), SRS tập trung vào việc dàn trải các phiên ôn tập theo thời gian. Nguyên lý then chốt của kỹ thuật này là thực hiện truy hồi thông tin ngay tại thời điểm trí nhớ bắt đầu suy giảm (forgetting point). Mỗi lần truy hồi thành công tại "điểm nút" này không chỉ khôi phục trí nhớ mà còn làm chậm đáng kể tốc độ quên lãng sau đó, giúp củng cố thông tin từ trí nhớ ngắn hạn sang trí nhớ dài hạn một cách bền vững.Hiệu quả của mô hình SRS trong hệ thống này được hiện thực hóa thông qua thuật toán SM-2 (SuperMemo-2). Đây là một mô hình toán học cho phép cá nhân hóa hoàn toàn lộ trình học tập dựa trên khả năng phản hồi của từng cá nhân. Khác với các cơ chế nhắc nhở cố định, SM-2 vận hành dựa trên sự biến thiên của chỉ số độ dễ (Easiness Factor - EF) — đại lượng đại diện cho độ phức tạp của một từ vựng đối với một người học cụ thể. Thông qua điểm số chất lượng phản hồi ($q$) từ 0 đến 5, hệ thống sẽ liên tục điều chỉnh $EF$ theo công thức:

$$EF' = EF + (0.1 - (5 - q) \times (0.08 + (5 - q) \times 0.02))$$

Dựa trên chỉ số này, khoảng cách giữa các lần ôn tập ($I$) sẽ được mở rộng dần theo cấp số nhân. Trong đó, lần ôn tập thứ nhất ($I_1$) thường là sau 1 ngày, lần thứ hai ($I_2$) là 6 ngày, và từ lần thứ $n$ trở đi, khoảng cách được tính theo công thức $I_n = I_{n-1} \times EF$. Một đặc điểm quan trọng của thuật toán là việc duy trì ngưỡng $EF \geq 1.3$ để đảm bảo khoảng cách không bị thu hẹp quá mức, tránh gây ra sự quá tải và nhàm chán cho người học. Sự linh hoạt của SM-2 nằm ở cơ chế "thử thách vừa đủ": nếu người học trả lời sai ($q < 3$), thuật toán sẽ lập tức thiết lập lại chu kỳ về 0, buộc từ vựng đó phải xuất hiện thường xuyên hơn để củng cố lại dấu vết trí nhớ. Ngược lại, những từ vựng dễ sẽ được đẩy xa hơn trong lịch trình, giúp tối ưu hóa thời gian và công sức. Sự kết hợp giữa quy luật tâm lý học của Ebbinghaus và tính chính xác của thuật toán SM-2 tạo nên một động lực học tập dựa trên sự tiến bộ thực tế, đặt nền móng cho việc xây dựng thói quen ghi nhớ bền vững trong hệ thống.

## 2.2 Lý thuyết Gamification và Động lực học tập

Dù các thuật toán như SM-2 giải quyết hiệu quả bài toán tối ưu hóa nhận thức, nhưng việc duy trì một lộ trình học tập kỷ luật trong dài hạn thường vấp phải rào cản lớn về tâm lý học hành vi. Gamification, hay việc vận dụng các thành phần và cơ chế của trò chơi vào bối cảnh phi trò chơi, đóng vai trò là chất xúc tác để "nội tại hóa" những nhiệm vụ sư phạm vốn dĩ khô khan. Thay vì chỉ dựa vào ý chí tự thân, hệ thống tận dụng các cấu trúc tương tác để kích thích hệ thống phần thưởng trong não bộ, từ đó nâng cao mức độ tham gia tự nguyện của người dùng. Để đảm bảo tính bền vững và tránh hiện tượng "burnout" (kiệt sức) do áp lực thành tích, nền tảng lý luận của hệ thống được xây dựng dựa trên Thuyết Tự Quyết (Self-Determination Theory - SDT) của Deci và Ryan.

Theo SDT, động lực con người không phải là một khối thống nhất mà là một dải phổ từ động lực ngoại tại đến động lực nội tại. Sự chuyển dịch này chỉ xảy ra khi ba nhu cầu tâm lý cơ bản được thỏa mãn đồng thời. Trước hết, nhu cầu về Năng lực (Competence) được hiện thực hóa thông qua các vòng phản hồi tức thì. Trong hệ thống, điểm kinh nghiệm (XP) và cấp độ (Level) không đơn thuần là những chỉ số định lượng, mà chúng đóng vai trò là bằng chứng trực quan cho sự tăng trưởng kiến thức. Khi người học nhận thấy sự tịnh tiến rõ rệt của bản thân qua từng phiên học, cảm giác làm chủ (mastery) sẽ được củng cố, thúc đẩy họ tiếp tục đương đầu với những thử thách khó hơn.

Bên cạnh đó, nhu cầu về Tự chủ (Autonomy) được đề cao thông qua việc trao quyền kiểm soát lộ trình cho người dùng. Khác với các phương pháp giáo trình truyền thống mang tính áp đặt, việc cho phép người học tự xây dựng bộ từ vựng cá nhân và lựa chọn linh thú đồng hành giúp biến quá trình học từ một nhiệm vụ bắt buộc thành một hành động tự nguyện đầy trách nhiệm. Sự gắn kết này càng trở nên sâu sắc hơn nhờ nhu cầu về Gắn kết (Relatedness), nơi các bảng xếp hạng công khai tạo ra một bối cảnh xã hội có tính cạnh tranh lành mạnh. Việc đặt sự tiến bộ cá nhân vào tương quan với cộng đồng không chỉ tạo ra động lực thúc đẩy mà còn mang lại cảm giác thuộc về một nhóm những người cùng chí hướng.

Điểm khác biệt cốt lõi trong mô hình gamification của hệ thống chính là việc lồng ghép hệ thống nuôi dưỡng thú ảo (Pet System) như một công cụ gắn kết cảm xúc. Thay vì sử dụng cơ chế "streak" (chuỗi ngày học liên tiếp) vốn dễ tạo ra áp lực tâm lý tiêu cực khi bị gián đoạn, hệ thống tập trung vào sự phát triển tích lũy của sinh vật ảo. Khi thú cưng sở hữu khả năng tiến hóa dựa trên nỗ lực của chủ nhân, chúng trở thành hiện thân cho thành quả lao động trí tuệ của người học. Sự chuyển dịch từ việc học "để không mất chuỗi" sang học "để nuôi dưỡng một sự sống ảo" là chìa khóa để duy trì thói quen bền vững, giúp người dùng tìm thấy ý nghĩa và niềm vui tự thân trong việc chinh phục ngôn ngữ mới.

## 2.3 Các công trình và ứng dụng liên quan

Những nguyên tắc học tập như lặp lại có khoảng cách (SRS) và thực hành truy hồi đã được nhiều ứng dụng học ngoại ngữ khai thác thành công, tạo ra những sản phẩm được hàng triệu người sử dụng. Tuy nhiên, qua phân tích sâu các ứng dụng phổ biến, vẫn tồn tại những khoảng trống nghiên cứu và thực tiễn đáng kể.

Một ví dụ tiêu biểu là **Duolingo**, ứng dụng ngôn ngữ phổ biến nhất hiện nay. Duolingo kết hợp hệ thống điểm kinh nghiệm (XP), streak và bảng xếp hạng để khuyến khích người học quay lại hằng ngày. Cách tiếp cận này giúp duy trì thói quen học tập rất tốt, đặc biệt đối với những người mới bắt đầu. Tuy nhiên, từ góc độ tâm lý học hành vi, việc quá phụ thuộc vào streak tạo ra một loại động lực ngoại tại (external motivation) mang tính áp lực. Khi streak bị gián đoạn, người dùng dễ rơi vào trạng thái "burnout" và có xu hướng bỏ hẳn thay vì quay lại học [4]. Đây chính là **khoảng trống về sự thiếu hụt cơ chế gắn kết cảm xúc bền vững**: hệ thống thiên về thúc đẩy bằng con số hơn là xây dựng mối liên kết nội tại với tiến trình phát triển của người học.

Một hướng tiếp cận khác được thể hiện rõ ở **Quizlet**, nơi người học chủ động tạo ra các bộ flashcard cá nhân. Ưu điểm của Quizlet là tính cá nhân hóa cao, cho phép người học tập trung vào đúng chủ đề họ quan tâm [5]. Tuy nhiên, Quizlet chủ yếu đóng vai trò như một công cụ hỗ trợ học tập thuần túy, thiếu vắng các yếu tố Gamification chuyên sâu. Điều này dẫn đến **sự tách biệt giữa động lực nội tại và ngoại tại**: người học cần có tính tự giác cực cao để duy trì thói quen vì hệ thống không thỏa mãn đồng bộ ba nhu cầu tâm lý theo thuyết SDT (Năng lực, Tự chủ, và Sự gắn kết).

Các ứng dụng như **Anki** và **Memrise** lại đi sâu vào cơ chế SRS tự động, tối ưu khoảng cách ôn tập dựa trên hiệu suất ghi nhớ [6, 7]. Đây là các công cụ cực kỳ hiệu quả về mặt khoa học nhận thức, nhưng lại tồn tại một rào cản lớn về trải nghiệm người dùng (UX). Giao diện thường khô khan, thiếu yếu tố xã hội và hoàn toàn không có tính "truyện kể" (narrative). Người dùng phổ thông thường cảm thấy việc học với Anki giống như một nhiệm vụ hành chính hơn là một hành trình khám phá, dẫn đến việc thiếu **mối liên kết cảm xúc (emotional connection)** với dữ liệu mình đang học.

Từ việc phân tích trên, có thể tổng kết **ba khoảng trống nghiên cứu (Research Gaps)** chính mà các hệ thống hiện tại chưa giải quyết triệt để:

- **Thiếu cơ chế gắn kết cảm xúc lâu dài:** Các hệ thống hiện có dựa quá nhiều vào streak (áp lực ngắn hạn) thay vì các cơ chế nuôi dưỡng/chăm sóc (như sinh vật ảo) gắn liền với tiến trình học để tạo sự gắn bó tự nhiên.
- **Sự mất cân bằng trong Gamification:** Phần lớn ứng dụng chỉ tập trung vào một vài yếu tố bề nổi (XP, Leaderboard) mà chưa triển khai hài hòa cả ba yếu tố của Thuyết Tự Quyết (SDT) để chuyển hóa động lực từ bên ngoài vào bên trong.
- **Sự thiếu hụt mô hình kết hợp tối ưu SRS + Gamification + Emotional Engagement:** Chưa có hệ thống nào kế thừa được thuật toán SRS mạnh mẽ của Anki, tính cá nhân hóa của Quizlet, và sự lôi cuốn của Duolingo nhưng lại loại bỏ được áp lực tiêu cực từ streak và sự khô khan trong giao diện.

Đây chính là điểm cốt lõi mà hệ thống hướng tới: xây dựng một "hành trình" có mục tiêu rõ ràng, nơi người học quay lại mỗi ngày không chỉ để hoàn thành bài tập mà còn để chăm sóc và nâng cấp thú cưng của mình. Việc biến sự tiến bộ trong kiến thức thành sự tiến hóa của sinh vật ảo giúp cụ thể hóa nhu cầu về năng lực (Competence) và tạo ra sự gắn bó lâu dài, khắc phục các hạn chế về động lực mà những ứng dụng đi trước gặp phải.

## 2.4 Công nghệ sử dụng trong hệ thống

Việc lựa chọn nền tảng công nghệ cho hệ thống được thực hiện theo tiêu chí thực tiễn: đáp ứng hiệu năng và độ trễ thấp cho thao tác người dùng nhiều tương tác, hỗ trợ mô hình phát triển an toàn, dễ kiểm thử và dễ bảo trì, có khả năng mở rộng (scale) khi cần phục vụ số lượng lớn người dùng, có hệ sinh thái công cụ và tài liệu phong phú để giảm rủi ro triển khai. Trên cơ sở đó, bộ công nghệ trọng tâm gồm **ASP.NET Core** (backend) + **Entity Framework Core** (ORM), **ReactJS** (frontend) kết hợp **Vite** và **TypeScript**, và **SQL Server** cho lưu trữ quan hệ.

### 2.4.1 ASP.NET Core

Lựa chọn ASP.NET Core xuất phát từ yêu cầu kết hợp hiệu năng thực tế, môi trường vận hành tin cậy và năng suất phát triển. So với Node.js + Express, ASP.NET Core cung cấp mô hình lập trình kiểu tĩnh (C#) và công cụ IDE mạnh (Visual Studio/VS Code) giúp phát hiện lỗi sớm và refactor an toàn; so với Django, .NET dễ tối ưu để đạt hiệu năng cực thấp cho các endpoint API có tải cao; so với Spring Boot, ASP.NET Core thường nhẹ hơn ở cấu hình khởi tạo cho hệ thống tầm trung và có cú pháp ngắn gọn hơn khi viết Web API hiện đại. Vì cần xử lý nhiều yêu cầu API (quiz, lưu tiến trình, tính toán SRS, push notification, real-time challenge, ...), một nền tảng có hiệu năng cao kèm khả năng phát triển nhanh là ưu tiên.

ASP.NET Core là kết quả tái thiết lại toàn bộ kiến trúc ASP.NET truyền thống, được phát hành chính thức cùng .NET Core từ giữa thập niên 2010 và ổn định từ 2016 trở về sau; mục tiêu chính là đa nền tảng (Windows/Linux/macOS), mô-đun hóa, và tối ưu cho triển khai cloud/container. Từ phiên bản ban đầu đến các bản .NET 6/7/8, đội ngũ .NET liên tục gia tăng tối ưu hiệu năng, bổ sung Minimal APIs, cải tiến GC và JIT, cùng các tiện ích cho microservices, observability và bảo mật.

Để hiểu vì sao ASP.NET Core phù hợp với hệ thống, cần làm rõ một số thành tố kỹ thuật then chốt:

- **Kestrel và mô hình hosting**: Kestrel là web server gốc của ASP.NET Core, được thiết kế cho hiệu năng cao, hỗ trợ HTTP/1.1, HTTP/2, HTTP/3 và WebSockets; Kestrel có thể

chạy trực tiếp hoặc kết hợp với reverse proxy (IIS/nginx) trong môi trường production, cho phép tối ưu theo yêu cầu tải và bảo mật. Nhờ Kestrel, các endpoint API có thể đạt throughput và latency rất tốt — điều quan trọng cho các API quiz/point updates cần phản hồi tức thì.

- **Middleware pipeline**: ASP.NET Core sử dụng pipeline middleware để xử lý luồng request–response theo chuỗi các thành phần (logging, authentication, authorization, validation, caching, compression). Kiến trúc này giúp tách rõ trách nhiệm, dễ chèn các logic xuyên suốt (ví dụ: rate limiting, global error handling) và tối ưu hoá từng lớp xử lý — rất hữu dụng cho việc áp dụng chính sách caching/điều phối cho các endpoint SRS hay content tĩnh.
- **Dependency Injection (DI) tích hợp**: DI được tích hợp sẵn, hỗ trợ các scope (singleton, scoped, transient), giúp cấu trúc Controller–Service–Repository rõ ràng. Điều này làm tăng testability (unit/integration tests) và giảm chi phí bảo trì khi mở rộng các tính năng như quy luật phần thưởng, logic tiến hóa thú cưng, hay đặt thời gian ôn tập cho SRS.
- **Asynchronous programming và I/O không đồng bộ**: C# / .NET có async/await và một runtime với thread pool, I/O không đồng bộ tốt — phù hợp cho xử lý nhiều request đồng thời (high concurrency) mà không phải thêm nhiều thread, tối ưu tài nguyên máy chủ khi phục vụ nhiều người dùng truy cập cùng lúc.
- **Bảo mật và Observability**: Hệ sinh thái ASP.NET Core cung cấp middleware cho authentication (JWT, cookie), ASP.NET Core Identity, Data Protection API, health checks và logging tích hợp, thuận lợi để hiện thực hoá chính sách bảo mật và giám sát sản phẩm trong môi trường production.
- **Tích hợp EF Core và quản lý schema**: Entity Framework Core hỗ trợ code-first, migrations, change tracking, compiled queries và raw SQL khi cần tối ưu, giúp quản lý schema tiến hoá theo phát triển tính năng. EF Core cân bằng được năng suất lập trình với khả năng tối ưu query khi cần.

Các đặc tính trên trực tiếp giải quyết yêu cầu nghiệp vụ của đề tài: API latency thấp cho trải nghiệm quiz mượt; background services cho lịch SRS và push notification; DI + unit tests đảm bảo logic phần thưởng dễ kiểm thử; EF Core giúp quản lý tiến trình học và migration khi thay đổi schema.

### 2.4.2 ReactJS

ReactJS được lựa chọn làm nền tảng frontend nhờ khả năng tạo giao diện người dùng tương tác mượt mà, cộng đồng phát triển lớn và hệ sinh thái phong phú. So với Angular (có đường cong học tập cao, nặng nề cho dự án vừa) và Vue (thân thiện nhưng hệ sinh thái nhỏ hơn), ReactJS đạt sự cân bằng giữa tính linh hoạt, hiệu năng và khả năng mở rộng. Điều này đặc biệt quan trọng với hệ thống, nơi UI cần liên tục cập nhật tiến trình học, điểm thưởng và các hiệu ứng thú cưng theo thời gian thực.

ReactJS được Meta (Facebook) giới thiệu năm 2013 với mục tiêu giải quyết vấn đề hiệu năng khi cập nhật giao diện động. Nó mang đến khái niệm Virtual DOM và component-based architecture, thay đổi cách xây dựng ứng dụng web so với phương pháp truyền thống. Qua nhiều phiên bản, ReactJS bổ sung Hooks, Concurrent Mode, Suspense, giúp quản lý trạng thái và side-effect đơn giản, hiệu quả hơn.

Để tối ưu trải nghiệm phát triển, hệ thống kết hợp ReactJS với Vite – công cụ build thế hệ mới hỗ trợ khởi động dự án cực nhanh, HMR (Hot Module Replacement) mượt mà – và

TypeScript để bổ sung kiểm tra kiểu tĩnh, giảm lỗi runtime và cải thiện khả năng bảo trì trong dự án dài hạn. Ngoài ra, Tailwind CSS được sử dụng để tăng tốc độ thiết kế và triển khai giao diện, nhờ cú pháp tiện lợi theo dạng utility-first và khả năng tuỳ biến cao.

- **Component-based architecture**: React khuyến khích tách UI thành components có trạng thái và props; điều này giúp xây dựng library UI (đều cho màn học, pet UI, progress bar) dễ tái sử dụng, test và bảo trì.
- **Virtual DOM và reconciliation**: Virtual DOM cho phép tính toán diff giữa state cũ và mới, từ đó cập nhật DOM thực tế chỉ ở những phần cần thiết — giảm chi phí repaint và tăng hiệu năng trên các màn có animation/interactive-heavy.
- **Hooks**: Hooks (useState, useEffect, useReducer, custom hooks) cung cấp API đơn giản để quản lý trạng thái và side-effect (ví dụ đồng bộ hóa progress với server, debounce input khi trả lời quiz).
- **TypeScript**: TypeScript thêm kiểm tra kiểu tĩnh, interface, generics giúp phát hiện lỗi sớm, cải thiện autocompletion, refactor an toàn — đặc biệt hữu ích khi codebase phức tạp có nhiều state, kiểu response API và mô hình dữ liệu học.
- **Vite**: Vite tạo môi trường phát triển gần như tức thời (fast cold start) và HMR rất mượt, rút ngắn vòng lặp develop–test, hỗ trợ tăng tốc độ phát triển tính năng UI (thú cưng, animation, transition). Vite cũng hỗ trợ build production (Rollup) để tối ưu tree-shaking và bundle size.
- **Tailwind CSS**: Tailwind CSS áp dụng triết lý utility-first, cung cấp sẵn nhiều lớp tiện ích cho layout, spacing, typography, color, animation; cho phép xây dựng UI thống nhất, hiện đại mà không cần viết nhiều CSS thủ công. Tailwind hỗ trợ tốt việc responsive và dark mode, đồng thời dễ dàng tuỳ chỉnh theme để đồng bộ với phong cách thiết kế riêng của hệ thống.

Việc sử dụng ReactJS cùng Vite, TypeScript và Tailwind CSS giúp hệ thống xây dựng giao diện hiện đại, tốc độ phản hồi nhanh, linh hoạt trong thiết kế và dễ mở rộng trong tương lai khi bổ sung thêm mini-game, dashboard nâng cao hay các chế độ học mới.

### 2.4.3 SQL Server

SQL Server được lựa chọn làm hệ quản trị cơ sở dữ liệu quan hệ vì khả năng tích hợp liền mạch với hệ sinh thái .NET, hỗ trợ ACID transaction và cung cấp công cụ quản trị trực quan thông qua SQL Server Management Studio. Đây là giải pháp ổn định, được nhiều doanh nghiệp sử dụng, đảm bảo tính toàn vẹn dữ liệu – yếu tố đặc biệt quan trọng với hệ thống học tập, nơi dữ liệu tiến trình và thành tích người học cần được bảo toàn chính xác.

Từ phiên bản đầu tiên năm 1989 đến nay, SQL Server đã phát triển với nhiều cải tiến như columnstore index, Always On Availability Groups, tích hợp Machine Learning Services, và hỗ trợ triển khai trên đám mây Azure. So với MySQL (phổ biến nhưng hạn chế về tính năng nâng cao) và PostgreSQL (mạnh về chuẩn SQL nhưng yêu cầu cấu hình thủ công nhiều hơn), SQL Server mang đến sự cân bằng giữa hiệu năng, khả năng quản trị và tính tiện lợi khi triển khai trong môi trường sản xuất.

### 2.4.4 Đánh giá chiến lược

Kết luận, tổ hợp **ASP.NET Core + EF Core** (backend) và **React + Vite + TypeScript** (frontend) với **SQL Server** ở tầng dữ liệu là một lựa chọn cân bằng giữa hiệu suất hệ thống, năng suất phát triển và khả năng vận hành/ mở rộng. Các đặc tính kỹ thuật của ASP.NET Core (Kestrel, middleware, DI, async model, background services) trực tiếp giải quyết những yêu cầu nghiệp vụ của hệ thống; trong khi tổ hợp frontend đảm bảo trải nghiệm mượt mà, dễ mở rộng UI và tốc độ phát triển. Việc chọn stack này không chỉ phục vụ mục tiêu đồ án mà còn thuận lợi cho việc mở rộng thành sản phẩm thực tế hoặc đưa vào môi trường production khi cần.

# Chapter 3

# Hệ thống hỗ trợ học từ vựng tiếng Anh kết hợp Gamification

Chương này trình bày quá trình phân tích và thiết kế hệ thống — một ứng dụng học từ vựng trực tuyến được phát triển dựa trên các nguyên lý học tập và gamification đã trình bày ở Chương 2. Mục tiêu của chương là xác định một cách rõ ràng các yêu cầu chức năng và phi chức năng, từ đó xây dựng mô hình kiến trúc, luồng xử lý và cơ sở dữ liệu.

Phân tích yêu cầu đóng vai trò trung gian quan trọng, chuyển hoá những khái niệm lý thuyết như *spaced repetition*, *retrieval practice* và các yếu tố trò chơi hoá thành tập hợp tính năng cụ thể, có thể triển khai được về mặt kỹ thuật. Trên cơ sở đó, phần thiết kế hệ thống sẽ tiếp tục khái quát kiến trúc tổng thể, trình bày các mô hình UML (Use Case, Activity, Sequence, Class, State), thiết kế cơ sở dữ liệu quan hệ và thiết kế giao diện đóng vai trò như "bản thiết kế" (blueprint) cho toàn bộ quá trình phát triển phần mềm.

## 3.1 Phân tích yêu cầu

### 3.1.1 Yêu cầu chức năng

Hệ thống được thiết kế để phục vụ hai nhóm đối tượng chính: **người học** và **quản trị viên**, với các chức năng đáp ứng đầy đủ nhu cầu học tập, theo dõi tiến trình và quản lý hệ thống.

**Đối với người học:**

- **Quản lý tài khoản:** Đăng ký, đăng nhập, cập nhật thông tin cá nhân và bảo vệ tài khoản.
- **Quản lý nội dung học:** Tự tạo, chỉnh sửa và xoá bộ từ vựng cá nhân; phân nhóm từ để học hoặc chia sẻ với cộng đồng.
- **Học tập và ôn luyện:** Lựa chọn bộ từ (cá nhân hoặc hệ thống cung cấp), tham gia các phiên học dưới dạng quiz đa dạng, và ôn tập dựa trên lịch nhắc tự động được tối ưu hoá theo nguyên tắc SRS.
- **Gamification và động lực học tập:** Nhận điểm thưởng, mở khoá phần thưởng, nâng cấp thú ảo, duy trì streak hằng ngày và theo dõi tiến trình học tập trực quan.
- **Thông báo và nhắc nhở:** Nhận thông báo về phiên ôn tập hoặc phần thưởng mới để duy trì thói quen học đều đặn.

**Đối với quản trị viên:**

- **Quản lý người dùng:** Theo dõi danh sách người dùng, điều chỉnh quyền truy cập, khoá hoặc mở tài khoản khi cần thiết.
- **Quản lý kho nội dung học:** Duy trì bộ từ chuẩn của hệ thống, thêm hoặc chỉnh sửa dữ liệu từ vựng.
- **Quản lý cơ chế thưởng:** Thêm mới hoặc tinh chỉnh hệ thống phần thưởng, thú ảo để tối ưu trải nghiệm gamification.
- **Báo cáo và phân tích:** Trích xuất dữ liệu thống kê phục vụ đánh giá hiệu quả học tập và tối ưu vận hành hệ thống.

### 3.1.2 Yêu cầu phi chức năng

Các yêu cầu phi chức năng được xây dựng nhằm bảo đảm hệ thống vận hành ổn định, bảo mật và mang lại trải nghiệm người dùng tốt:

- **Hiệu năng:** Thời gian phản hồi của mỗi thao tác dưới 3 giây để duy trì trải nghiệm mượt mà, đặc biệt đối với các quiz thời gian thực.
- **Bảo mật:** Mật khẩu được mã hoá an toàn; API bảo vệ bằng cơ chế xác thực JWT; dữ liệu người dùng tuân thủ nguyên tắc bảo mật và quyền riêng tư.
- **Khả năng mở rộng:** Hệ thống được thiết kế module hoá, cho phép bổ sung bộ từ mới, loại quiz mới hoặc phần thưởng mà không gây gián đoạn dịch vụ.
- **Khả năng sử dụng:** Giao diện thân thiện, trực quan, hỗ trợ người dùng mới bắt đầu dễ dàng làm quen, đồng thời tối ưu cho cả thiết bị di động và máy tính.

## 3.2 Thiết kế hệ thống

### 3.2.1 Kiến trúc hệ thống tổng thể

Kiến trúc hệ thống được xây dựng theo mô hình Client–Server ba lớp (Three-Tier Architecture), một kiến trúc phổ biến và đã được kiểm chứng trong các ứng dụng web quy mô vừa và lớn. Mô hình này tách biệt rõ ràng các thành phần Giao diện người dùng (Presentation Layer), Xử lý nghiệp vụ (Application/Business Logic Layer) và Lưu trữ dữ liệu (Data Layer), từ đó đảm bảo khả năng bảo trì, mở rộng và phân tách trách nhiệm giữa các nhóm phát triển.

**Lớp Giao diện người dùng (Frontend)**: Lớp này được hiện thực bằng ReactJS kết hợp Vite, TypeScript và TailwindCSS, chịu trách nhiệm hiển thị toàn bộ nội dung học tập và cung cấp trải nghiệm tương tác mượt mà cho người dùng cuối. ReactJS với kiến trúc component-based cho phép chia nhỏ giao diện thành các thành phần tái sử dụng, từ đó giảm thiểu độ phức tạp trong phát triển và bảo trì. Vite mang lại khả năng biên dịch nhanh, hỗ trợ Hot Module Replacement (HMR), giúp nhóm phát triển quan sát thay đổi giao diện gần như theo thời gian thực. TailwindCSS giúp định kiểu giao diện một cách nhất quán, dễ dàng tuỳ biến theo thiết kế gamification của hệ thống.

Frontend giao tiếp với backend thông qua các API RESTful, thực hiện các thao tác chính như:

- Hiển thị danh sách bộ từ, nội dung từ vựng và lịch sử học tập.

- Thực hiện phiên học (learning session) với nhiều dạng quiz như điền từ, trắc nghiệm, nghe – viết.
- Hiển thị tiến trình học, điểm thưởng, bảng xếp hạng và trạng thái thú ảo theo cơ chế gamification.
- Gửi yêu cầu đăng nhập, đăng ký, cập nhật thông tin cá nhân và nhận thông báo nhắc nhở học tập.

**Lớp Xử lý nghiệp vụ (Backend)**: Lớp này được triển khai bằng ASP.NET Core Web API, đóng vai trò là trung tâm xử lý nghiệp vụ và điều phối luồng dữ liệu giữa giao diện và cơ sở dữ liệu. Toàn bộ các chức năng chính của hệ thống như quản lý người dùng, quản lý từ vựng, xử lý tiến trình học, tính toán điểm thưởng, nâng cấp thú ảo, và gửi thông báo đều được hiện thực tại lớp này.

Một số đặc trưng nổi bật của lớp backend:

- **RESTful API:** Cung cấp endpoint rõ ràng, chuẩn hoá, hỗ trợ nhiều client (web, mobile) tiêu thụ cùng một nguồn dữ liệu.
- **Middleware Pipeline:** Cho phép xây dựng chuỗi xử lý request–response có thể mở rộng, ví dụ thêm middleware xác thực JWT, logging, xử lý lỗi tập trung.
- **Entity Framework Core:** Đóng vai trò ORM, ánh xạ dữ liệu giữa mô hình đối tượng C# và bảng trong cơ sở dữ liệu theo phương pháp Code-First.
- **Bảo mật:** Ứng dụng cơ chế xác thực JWT Authentication để bảo vệ API, quản lý phân quyền theo vai trò (người dùng, quản trị viên).

Kiến trúc backend đảm bảo có thể dễ dàng mở rộng trong tương lai, ví dụ thêm microservice riêng cho xử lý thống kê hoặc tích hợp dịch vụ AI (nhận diện giọng nói, gợi ý từ vựng thông minh).

**Lớp Dữ liệu (Database)**: Lớp dữ liệu sử dụng SQL Server làm hệ quản trị cơ sở dữ liệu quan hệ, chịu trách nhiệm lưu trữ toàn bộ thông tin của hệ thống như:

- Hồ sơ người dùng, thông tin đăng nhập và phân quyền.
- Bộ từ vựng, thông tin định nghĩa, phát âm và ví dụ minh họa.
- Tiến trình học tập, lịch sử làm bài, kết quả trả lời quiz.
- Thông tin phần thưởng, thú ảo, trạng thái tiến hoá.
- Bảng thông báo, sự kiện và nhắc nhở học tập.

Dữ liệu được thiết kế theo mô hình quan hệ, áp dụng các ràng buộc toàn vẹn (khoá chính, khoá ngoại, unique constraint) để giảm thiểu lỗi logic. Hệ thống cũng chuẩn bị cho khả năng mở rộng bằng cách tách biệt các bảng liên quan đến tracking (nhật ký học tập) nhằm tối ưu hiệu năng truy vấn thống kê và báo cáo.

Ba lớp trên phối hợp nhịp nhàng thông qua giao tiếp client–server chuẩn HTTP/HTTPS, tạo nên hệ thống vừa trực quan, vừa mạnh mẽ. Kiến trúc ba lớp không chỉ tối ưu khả năng bảo trì và mở rộng, mà còn tạo điều kiện dễ dàng cho việc triển khai trên các nền tảng hạ tầng khác nhau (on-premises hoặc cloud).

**Kiến trúc Frontend chi tiết — Component-Based Architecture**

Ngoài mô tả tổng quát ở trên, lớp Frontend được tổ chức theo mô hình component-based architecture, tối ưu cho khả năng tái sử dụng, bảo trì và mở rộng tính năng. Cấu trúc thư mục Frontend được sắp xếp theo nguyên tắc Clean Architecture, với sự phân tách rõ ràng giữa các lớp:

**1. Components Layer**: Chứa các reusable UI components như buttons, inputs, modals, cards. Các components này có tính độc lập cao, dễ dàng được tích hợp vào các trang khác nhau mà không cần chỉnh sửa logic.

**2. Features/Pages Layer**: Tổ chức theo từng tính năng chính (Learning, VocabularySets, Pet, User Profile). Mỗi feature bao gồm:

- Pages: Các màn hình chính (LearningPage, VocabularySetPage, PetPage).
- Components: Các component cấp cao dùng riêng cho feature này.
- Hooks: Custom React hooks quản lý logic đặc thù của feature (useVocabularySet, useLearningSession, usePet).
- Services: API integration layer cho feature đó.

**3. Services/Hooks Layer**: Quản lý giao tiếp với backend:

- Services: Axios clients cho các API endpoints (UserService, VocabularyService, LearningService, PetService). Mỗi service tương ứng với một nhóm endpoint trên backend.
- Custom Hooks: useAuth(), useLearningSession(), usePet(), v.v. để encapsulate complex state logic và side effects.
- SignalR Integration: Real-time notification service cho reminder học tập và thông báo thú cưng.

**4. State Management Layer (Redux)**: Quản lý global state:

- Reducers: authReducer, userReducer, vocabularyReducer, petReducer, learningReducer.
- Actions: Các action creators để dispatch state updates (login, logout, updatePet, recordLearningSession).
- Selectors: Helper functions để select data từ Redux store một cách consistent.

**5. Utilities/Types Layer**: Hỗ trợ công cụ và type definitions:

- Utilities: Helper functions cho validation, formatting, calculations (calculateSRS, formatDate, generatePetImageUrl).
- Types/Interfaces: TypeScript definitions cho User, VocabularySet, LearningSession, Pet, Achievement (đảm bảo type safety).
- Constants: Enum definitions, API endpoints, configuration constants.

Kiến trúc Frontend này đảm bảo:

- **Modularity**: Mỗi feature độc lập, có thể phát triển song song và test riêng.
- **Reusability**: Components reusable, hooks sharable giữa các features.

- **Testability**: Dễ unit test components (với Jest), integration test flows (với Playwright).
- **Maintainability**: Cấu trúc rõ ràng, dễ debug và thêm features mới.
- **Type Safety**: TypeScript ensures tất cả props, state, API responses đều được type-check tại compile time.

**Kiến trúc Backend chi tiết — Clean Architecture & Design Patterns**

Ngoài mô hình ba lớp tổng quát, lớp backend được thiết kế chi tiết theo Clean Architecture (Onion Architecture), một kiến trúc phần mềm tiên tiến nhằm tối ưu khả năng testability, maintainability và independence từ framework/library. Kiến trúc này tổ chức code thành các lớp đồng tâm, mỗi lớp có trách nhiệm riêng biệt.

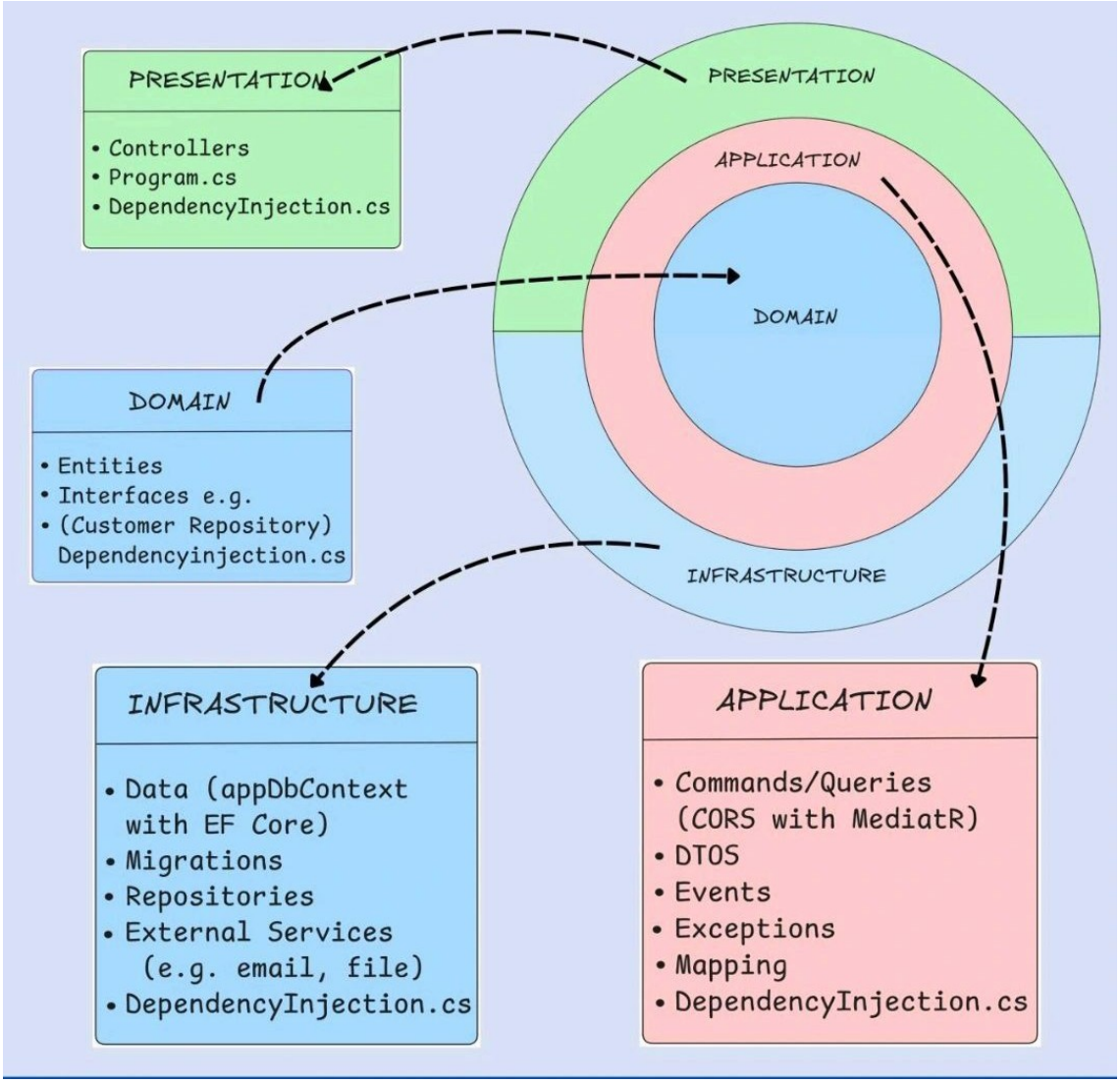


Figure 3.1: Kiến trúc Clean (Onion Architecture).

**Bốn Layers của Onion Architecture:**

**1. Domain Layer (WordSoul.Domain):** Lớp này chứa các entities và business rules cốt lõi, hoàn toàn độc lập với bất kỳ framework hay library nào. Nó bao gồm:

- **Entities**: 25 entities chính (User, Pet, VocabularySet, LearningSession, UserVocabularyProgress, VocabularyReviewHistory, Achievement, v.v.), biểu diễn các khái niệm nghiệp vụ.
- **Enums**: UserRole (User, Admin), PetType, DailyQuestType để định kiểu các trạng thái.
- **Value Objects**: Các object có tính chất bất biến, ví dụ Email, PhoneNumber.
- **Domain Services**: Các service chứa business logic không thuộc về entity cụ thể nào, ví dụ SRS scheduling algorithm.
- **Exceptions**: Custom exceptions cho domain-specific errors, giúp communicate lỗi rõ ràng.

**2. Application Layer (WordSoul.Application):** Lớp ứng dụng phối hợp giữa Domain và Infrastructure. Nó chứa:

- **Application Services** (20+ services): UserService, PetService, LearningSessionService, VocabularyService, v.v., chiết xuất business logic và điều phối các operation.
- **DTOs (Data Transfer Objects)**: UserDto, PetDto, LearningSessionDto, v.v. dùng để tách database models khỏi API responses, bảo vệ internal structure.
- **Service Interfaces**: IUserService, IPetService, ILearningSessionService, v.v., định nghĩa contracts mà Infrastructure layer phải implement, giúp decoupling.
- **SRS Algorithm**: Triển khai SM-2 algorithm trong Services/SRS/ folder, độc lập từ database details, chỉ phụ thuộc vào Domain.

**3. Infrastructure Layer (WordSoul.Infrastructure):** Lớp này implement từ các interfaces của Application layer và quản lý external services, database, v.v:

- **Repositories** (23 repositories): Mỗi entity có một repository riêng (UserRepository, PetRepository, VocabularySetRepository, v.v.), implement từ IRepository interface, cung cấp abstraction cho database access.
- **Unit of Work Pattern**: UnitOfWork.cs quản lý tất cả repositories, đảm bảo consistency của transactions và lazy-loading optimization. Khi một service gọi UnitOfWork, nó nhận được công cụ để truy cập bất kỳ repository nào mà không phải tạo instance trực tiếp.
- **DbContext**: WordSoulDbContext chứa DbSet cho tất cả entities, migration history, và cấu hình Entity Framework Core.
- **Persistence**: Migrations folder, database seeding, schema management.
- **External Services**: Integration với Cloudinary (upload images), email service, notification service.
- **Background Services**: Scheduled tasks như reminder notifications, daily quest reset.

**4. Presentation Layer (WordSoul.Api):** Lớp này là entry point cho tất cả requests:

- **Controllers**: API endpoints, nhận HTTP requests và dispatch tới application services. Ví dụ: UserController, PetController, LearningSessionController.

- **Middleware**: JWT authentication, error handling, logging.
- **Dependency Injection (DI) Configuration**: Program.cs cấu hình tất cả services, repositories, và DbContext vào DI container.
- **Filters**: Authorization filters, validation filters tùy chỉnh.
- **Helpers**: Utility functions cho response formatting, token generation.

**Key Design Patterns Triển khai:**

**1. Unit of Work Pattern:** UnitOfWork.cs quản lý tất cả 23 repositories qua lazy-loading properties. Khi một service cần repository, nó được khởi tạo một lần và tái sử dụng. Điều này giúp:

- Quản lý transaction atomicity: SaveChangesAsync() trên UnitOfWork đảm bảo tất cả changes được lưu cùng lúc hoặc không có gì được lưu.
- Giảm số lượng DbContext instances, tối ưu memory.
- Simplify dependency management: Services chỉ phụ thuộc vào IUnitOfWork, không phải 23 repositories riêng.

**2. Repository Pattern:** Mỗi entity có một repository implement từ interface. Ví dụ UserRepository implement IUserRepository với các methods: GetAllUsersAsync, GetUserByIdAsync, AddAsync, UpdateAsync, DeleteAsync. Điều này cho phép:

- Abstract database details khỏi business logic.
- Dễ unit test bằng cách mock repositories.
- Thay đổi persistence mechanism (từ SQL Server sang PostgreSQL) mà không ảnh hưởng application services.

**3. Dependency Injection:** ASP.NET Core built-in DI container, cấu hình trong Program.cs:

- Scoped dependencies: DbContext, UnitOfWork (một instance per HTTP request).
- Singleton dependencies: Configuration, ILogger factory.
- Transient dependencies: Stateless services.

Điều này giúp:

- Loose coupling giữa services.
- Dễ testing: mock dependencies trong unit tests.
- Centralized configuration của tất cả services.

**4. DTO (Data Transfer Object) Pattern:** Services trả về DTOs thay vì entities. Ví dụ UserService trả về UserDto, không phải User entity. Lợi ích:

- Bảo vệ internal structure: frontend không cần biết entity có những property nào, chỉ cần DTO schema.
- Chuyển đổi data: Entity có tất cả columns, nhưng DTO chỉ chứa fields cần thiết để gửi về client.

- Reusability: Một entity có thể được map sang nhiều DTOs khác nhau tùy context (list view vs detail view).

**5. Service Layer Pattern:** ApplicationServices (UserService, PetService, ...) chứa orchestration logic:

- Nhận requests từ Controllers.
- Gọi domain services, repositories để lấy data hoặc perform business logic.
- Transform entities thành DTOs.
- Return DTOs tới Controllers.

Sự phân tách này đảm bảo:

- Single Responsibility: Mỗi layer có một trách nhiệm rõ ràng.
- Testability: Unit test business logic mà không cần database, HTTP requests.
- Maintainability: Thay đổi database schema không ảnh hưởng application logic.
- Scalability: Dễ thêm features mới mà không phá vỡ existing code.

Tóm lại, kiến trúc hệ thống WordSoul được xây dựng trên nền tảng ba lớp, với frontend sử dụng Clean Architecture component-based pattern và backend triển khai đầy đủ Onion Architecture cùng các design patterns tiên tiến. Sự kết hợp này không chỉ đảm bảo khả năng bảo trì, mở rộng và kiểm thử, mà còn tạo ra một đơn vị phần mềm vững chắc và linh hoạt để triển khai các chức năng được xác định trong phần phân tích yêu cầu.

### 3.2.2 Sơ đồ Use Case

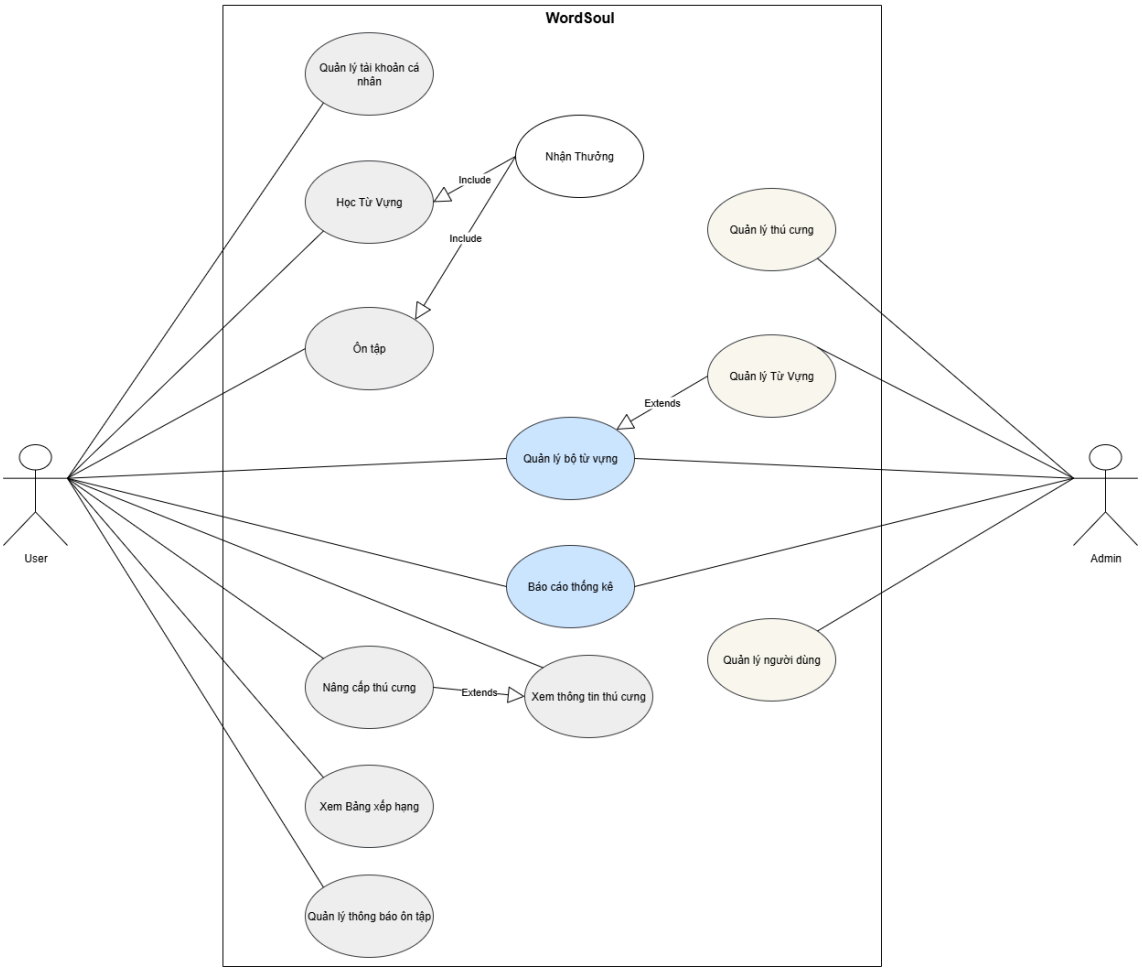


Figure 3.2: Sơ đồ Use Case của hệ thống.

Sơ đồ Use Case ở Hình 3.2 mô tả phạm vi chức năng tổng thể của hệ thống và thể hiện rõ mối quan hệ giữa hai tác nhân chính với các chức năng mà họ tương tác. Đây là công cụ quan trọng trong giai đoạn phân tích yêu cầu, giúp đảm bảo hệ thống được thiết kế đúng với nhu cầu người học và quản trị viên, đồng thời làm cơ sở cho việc đặc tả chi tiết và kiểm thử phần mềm.

**Các tác nhân (Actors)**: Hệ thống xác định hai nhóm người dùng chính, mỗi nhóm có vai trò riêng biệt:

- **Người học (User):** Là đối tượng trung tâm của hệ thống, thực hiện toàn bộ hoạt động học và ôn tập. Người học không chỉ tiêu thụ nội dung mà còn có khả năng cá nhân hoá trải nghiệm bằng cách tự tạo, chỉnh sửa và xoá bộ từ vựng cá nhân. Điều này giúp đáp ứng nhu cầu học đa dạng, từ học theo chương trình chuẩn đến học theo sở thích hoặc mục tiêu cá nhân.

- **Quản trị viên (Admin):** Đóng vai trò bảo đảm chất lượng và tính toàn vẹn của hệ

thống. Admin chịu trách nhiệm cập nhật kho dữ liệu từ vựng chuẩn, duy trì hệ thống phần thưởng và thú ảo, cũng như quản lý người dùng. Điều này giúp hệ thống vận hành ổn định và liên tục được cập nhật để người học luôn có nội dung mới.

**Các nhóm Use Case chính**: Các chức năng được nhóm theo luồng nghiệp vụ để phản ánh rõ vai trò của hệ thống:

- **Học và ôn tập từ vựng:** Bao gồm lựa chọn bộ từ, khởi tạo phiên học hoặc phiên ôn tập, làm các dạng quiz (điền khuyết, trắc nghiệm, nghe-viết). Đây là nhóm chức năng cốt lõi, hiện thực hoá các nguyên lý SRS và retrieval practice đã phân tích ở Chương 2.
- **Quản lý bộ từ cá nhân:** Cho phép người học tạo mới bộ từ, chỉnh sửa, xoá hoặc nhập dữ liệu từ có sẵn. Việc tách riêng nhóm chức năng này nhằm hỗ trợ cá nhân hoá, giúp người học làm chủ nội dung học.
- **Theo dõi tiến trình và nhận thông báo:** Người học có thể xem báo cáo thống kê (số từ đã học, tỉ lệ ghi nhớ), nhận thông báo nhắc nhở ôn tập và duy trì thói quen học đều đặn.
- **Gamification và phần thưởng:** Sau mỗi phiên học/ôn tập, hệ thống tự động xét thưởng (XP, thú ảo, phần thưởng đặc biệt) để duy trì động lực cho người học.
- **Quản lý thú ảo (Pet):** Người học có thể xem thông tin thú cưng và nâng cấp chúng khi đủ điểm thưởng. Việc tách use case "Nâng cấp Pet" dưới dạng *extend* từ "Xem thông tin Pet" bảo đảm tính điều kiện: người học chỉ có thể nâng cấp khi đã vào màn hình xem Pet và đáp ứng đủ yêu cầu.
- **Quản trị hệ thống:** Cho phép Admin tạo hoặc cập nhật từ vựng chuẩn, quản lý người dùng và quản lý danh mục phần thưởng, đảm bảo hệ thống vận hành liên tục và dữ liệu luôn chính xác.

**Các mối quan hệ Use Case**: Sơ đồ thể hiện rõ quan hệ **include** và **extend** để nhấn mạnh sự tái sử dụng và điều kiện thực thi:

- **Include:** Use case "Nhận thưởng" được include trong cả "Học từ vựng" và "Ôn tập từ vựng" vì đây là hành vi luôn xảy ra sau khi phiên học kết thúc.
- **Extend:** Use case "Nâng cấp Pet" extend từ "Xem thông tin Pet" để biểu diễn rằng chức năng nâng cấp chỉ xuất hiện khi người học đã mở thông tin Pet và đủ điều kiện nâng cấp.
- **Extend:** "Quản lý bộ từ cá nhân" extend từ "Quản lý từ vựng", bổ sung các chức năng dành riêng cho người học mà không ảnh hưởng đến kho dữ liệu chuẩn do Admin quản lý.

**Ý nghĩa và giá trị**: Việc xây dựng sơ đồ Use Case giúp định nghĩa ranh giới hệ thống, phân tách rõ trách nhiệm giữa User và Admin, đồng thời hỗ trợ phát hiện các tình huống đặc biệt cần xử lý (ví dụ: điều kiện nâng cấp Pet, thông báo nhắc nhở). Đây là bước quan trọng để đảm bảo giai đoạn thiết kế sau đó có cơ sở đầy đủ, hạn chế sai sót trong triển khai và tăng khả năng kiểm thử tự động.

### 3.2.3 Sơ đồ tuần tự

**Quy trình học, ôn từ vựng và nhận thưởng**

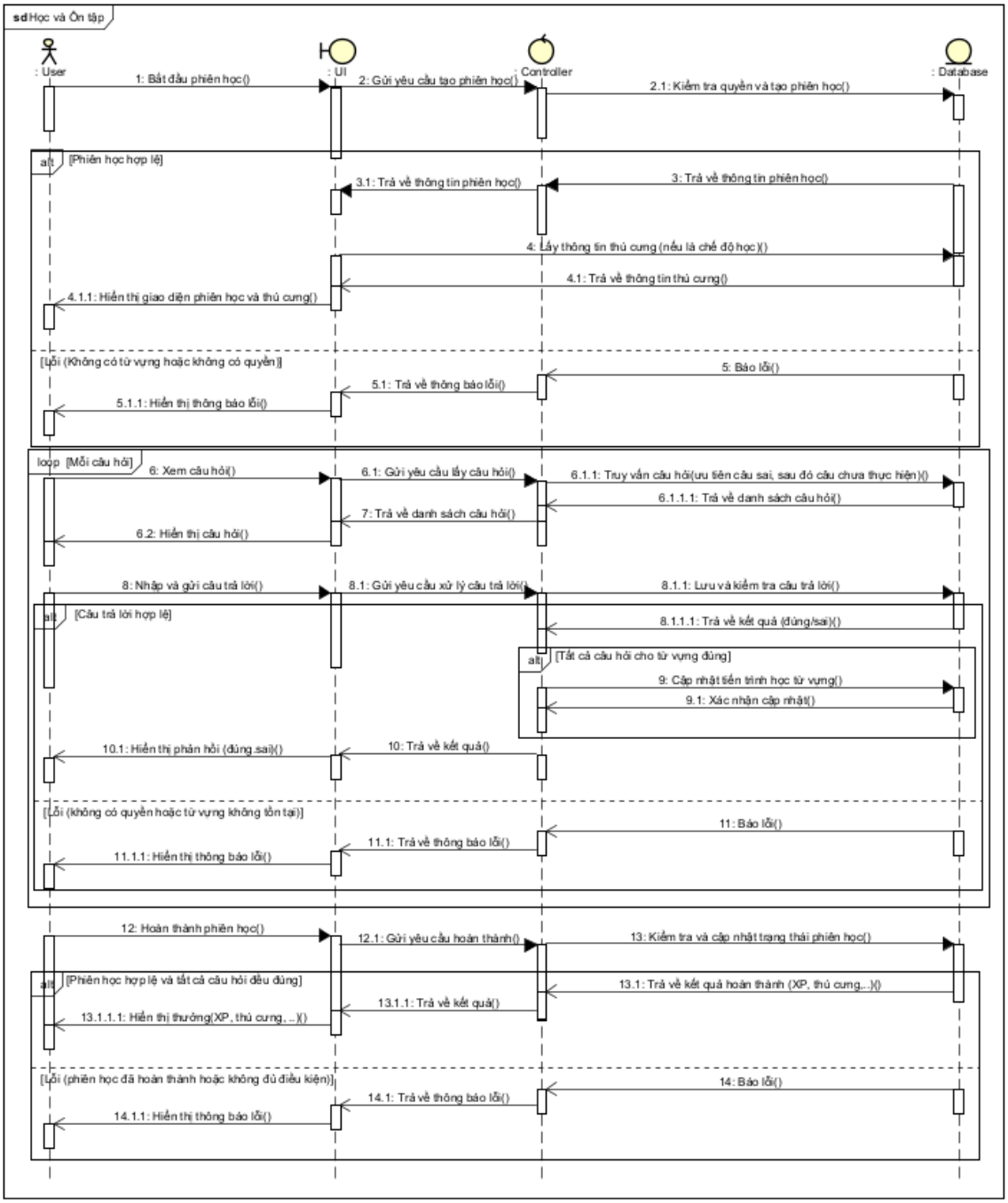


Figure 3.3: Sơ đồ tuần tự quy trình học, ôn tập từ vựng và nhận thưởng.

Sơ đồ tuần tự trên mô tả toàn bộ quá trình xử lý khi người dùng thực hiện một phiên học hoặc ôn tập từ vựng:

- **Khởi tạo phiên học**:
  - Người dùng từ giao diện (UI) chọn một bộ từ vựng và yêu cầu bắt đầu phiên học.
  - Controller tiếp nhận yêu cầu, tiến hành kiểm tra quyền truy cập (người dùng có sở hữu bộ từ vựng này không, còn hợp lệ không).
  - Nếu hợp lệ, Controller tạo mới một phiên học trong cơ sở dữ liệu (Database) và trả về thông tin phiên học; ngược lại, hệ thống phản hồi thông báo lỗi.
- **Chuẩn bị giao diện phiên học**:
  - Với chế độ học, hệ thống còn lấy thêm thông tin thú cưng với tỉ lệ ngẫu nhiên.
  - UI hiển thị giao diện phiên học, bao gồm bộ từ vựng cần học/ôn tập và thú cưng đồng hành (nếu có), nhằm tạo trải nghiệm học tập sinh động hơn.
- **Truy xuất và hiển thị câu hỏi**:
  - Người dùng gửi yêu cầu xem câu hỏi tiếp theo.
  - Controller truy vấn danh sách câu hỏi từ Database, với cơ chế ưu tiên câu sai trước, sau đó mới đến câu chưa từng thực hiện.
  - Database trả về danh sách câu hỏi, sau đó UI hiển thị cho người dùng.
- **Người dùng trả lời và xử lý đáp án**:
  - Khi người dùng nhập câu trả lời, UI gửi dữ liệu đến Controller để xử lý.
  - Controller lưu kết quả, kiểm tra đúng/sai và phản hồi ngay lập tức cho UI.
  - Nếu người dùng trả lời đúng toàn bộ câu hỏi liên quan đến một từ vựng, hệ thống cập nhật tiến trình học của từ đó (đánh dấu "đã ghi nhớ").
  - UI hiển thị kết quả từng câu hỏi, giúp người học nhận được phản hồi tức thì.
- **Hoàn tất phiên học**:
  - Khi người dùng hoàn thành toàn bộ câu hỏi, UI gửi yêu cầu kết thúc phiên học.
  - Controller kiểm tra và cập nhật trạng thái phiên học trong Database.
  - Hệ thống tính toán phần thưởng cho người dùng, bao gồm kinh nghiệm (XP) và phần thưởng thú cưng (nếu có).
  - UI hiển thị kết quả cuối cùng hoặc thông báo lỗi (nếu phiên học không hợp lệ).

**Ý nghĩa:** Sơ đồ này thể hiện rõ cơ chế học chủ động (active recall) với: Phản hồi tức thì sau mỗi câu trả lời, cập nhật tiến trình liên tục để người học theo dõi, gamification thông qua XP và thú cưng, giúp duy trì động lực. Điểm quan trọng là hệ thống không chỉ cung cấp nội dung học mà còn quản lý chu trình học - phản hồi - củng cố - thưởng, tạo ra trải nghiệm liền mạch, vừa khoa học vừa hấp dẫn.

**Quy trình tạo bộ từ vựng**

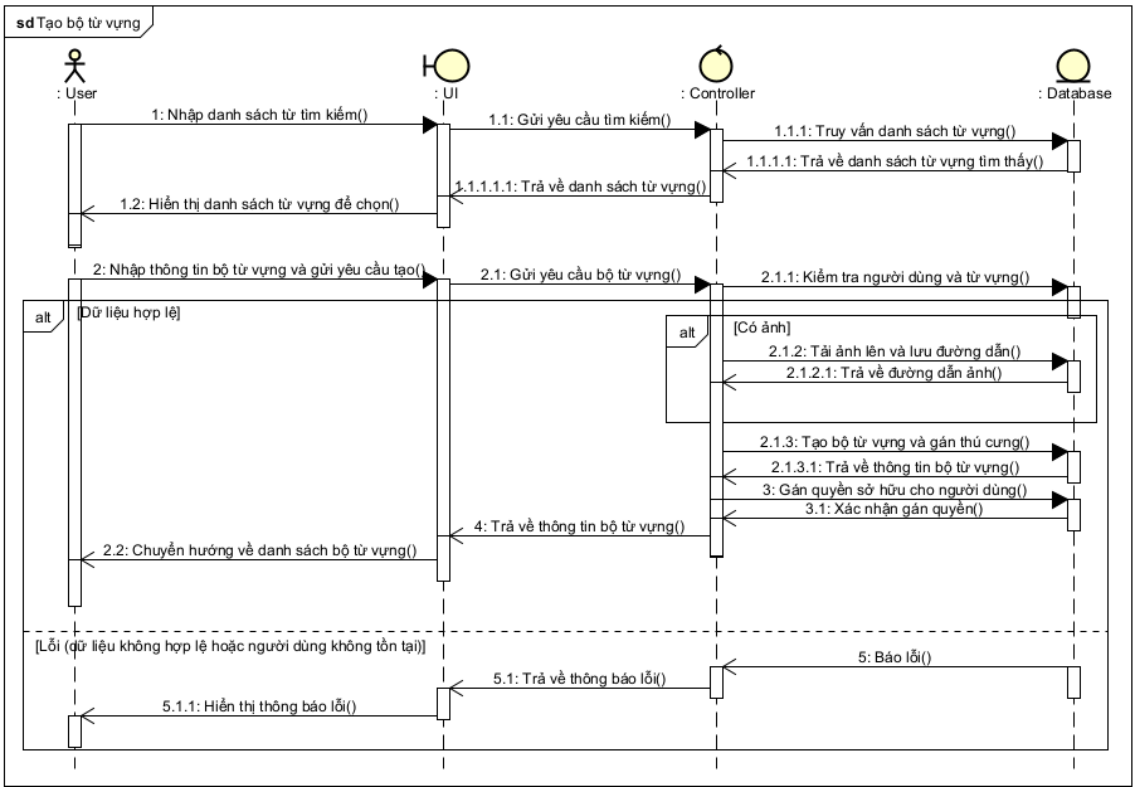


Figure 3.4: Sơ đồ tuần tự quy trình tạo bộ từ vựng.

Sơ đồ tuần tự trên mô tả chi tiết luồng xử lý khi người dùng tạo một bộ từ vựng mới:

- **Tìm kiếm từ vựng để thêm vào bộ từ**:
  - Người dùng nhập danh sách từ cần tìm qua giao diện.
  - UI gửi yêu cầu đến Controller, sau đó Controller truy vấn cơ sở dữ liệu (Database).
  - Database trả về danh sách từ vựng hợp lệ; UI hiển thị danh sách này để người dùng lựa chọn.
- **Nhập thông tin bộ từ và gửi yêu cầu tạo**:
  - Người dùng nhập thông tin chi tiết cho bộ từ, bao gồm: tiêu đề, mô tả, chủ đề, độ khó, ảnh minh họa (nếu có) và chế độ công khai.
  - UI gửi yêu cầu tạo bộ từ mới đến Controller.
- **Xử lý và kiểm tra dữ liệu**:
  - Controller tiến hành kiểm tra tính hợp lệ của dữ liệu và quyền của người dùng.
  - Nếu người dùng tải ảnh minh họa, hệ thống sẽ lưu ảnh và trả về đường dẫn để gắn vào bộ từ.

- **Tạo bộ từ và gán quyền sở hữu**:
  - Controller tạo mới bản ghi bộ từ trong Database, đồng thời gán một thú cưng mặc định đi kèm.
  - Hệ thống cũng gán quyền sở hữu bộ từ cho người dùng đã tạo, đảm bảo quản lý chính xác mối quan hệ chủ sở hữu – bộ từ.
- **Phản hồi kết quả**:
  - Nếu tạo thành công, UI nhận thông tin bộ từ mới và chuyển hướng người dùng về danh sách bộ từ cá nhân.
  - Nếu dữ liệu không hợp lệ hoặc người dùng không tồn tại, hệ thống trả về thông báo lỗi, UI hiển thị thông báo này.

**Ý nghĩa:** Sơ đồ thể hiện quy trình kiểm tra dữ liệu chặt chẽ trước khi ghi nhận vào hệ thống. Mỗi bộ từ được gắn với một chủ sở hữu hợp lệ, có thông tin đầy đủ và (nếu cần) kèm ảnh minh họa, đảm bảo tính cá nhân hóa cũng như quản lý hiệu quả. Cơ chế này giúp hệ thống duy trì dữ liệu nhất quán, tránh tình trạng bộ từ vựng không xác định hoặc trùng lặp.

### Quy trình nâng cấp thú cưng

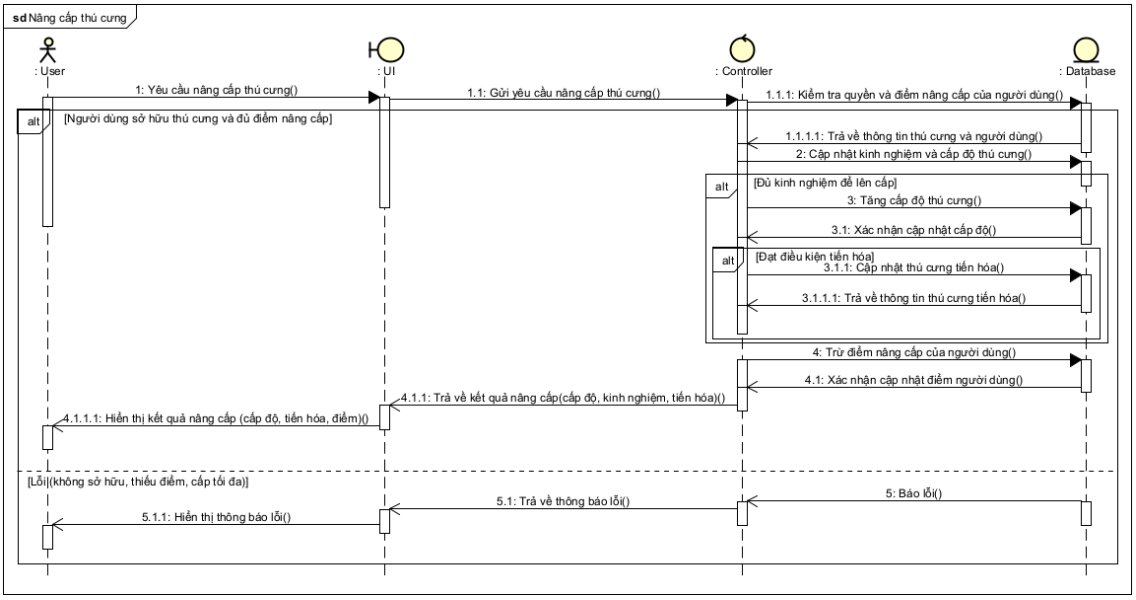


Figure 3.5: Sơ đồ tuần tự quy trình nâng cấp thú cưng.

Sơ đồ tuần tự trên mô tả chi tiết các bước khi người dùng thực hiện nâng cấp thú cưng:

- **Yêu cầu nâng cấp**:
  - Người dùng từ giao diện gửi yêu cầu nâng cấp một thú cưng mà họ đang sở hữu.
  - UI chuyển yêu cầu này đến Controller để xử lý.

- **Kiểm tra điều kiện**:
  - Controller kiểm tra trong cơ sở dữ liệu quyền sở hữu thú cưng và số điểm nâng cấp hiện có của người dùng.
  - Nếu không hợp lệ (không sở hữu thú cưng, thiếu điểm, hoặc thú cưng đã đạt cấp tối đa), hệ thống trả về thông báo lỗi ngay lập tức.
- **Cập nhật kinh nghiệm và xét điều kiện nâng cấp**:
  - Nếu hợp lệ, Controller tiến hành cập nhật kinh nghiệm cho thú cưng.
  - Trường hợp thú cưng có đủ kinh nghiệm để lên cấp, hệ thống thực hiện tăng cấp độ và xác nhận cập nhật vào cơ sở dữ liệu.
  - Nếu thú cưng đạt điều kiện tiến hoá, hệ thống thực hiện cập nhật trạng thái tiến hoá và lưu lại thông tin mới.
- **Trừ điểm nâng cấp và xác nhận**:
  - Hệ thống trừ số điểm nâng cấp của người dùng tương ứng với hành động vừa thực hiện.
  - Controller xác nhận cập nhật dữ liệu điểm nâng cấp của người dùng trong Database.
- **Trả về kết quả**:
  - Sau khi tất cả cập nhật được ghi nhận, Controller trả về thông tin mới của thú cưng, bao gồm: cấp độ, trạng thái tiến hoá (nếu có), kinh nghiệm và điểm nâng cấp còn lại.
  - UI hiển thị kết quả nâng cấp cho người dùng.
  - Nếu có lỗi xảy ra ở bất kỳ bước nào (ví dụ dữ liệu không hợp lệ), UI sẽ hiển thị thông báo lỗi tương ứng.

**Ý nghĩa:** Sơ đồ làm rõ quá trình nâng cấp thú cưng được kiểm tra nhiều lớp điều kiện để tránh sai lệch dữ liệu. Cơ chế vừa bảo đảm đồng bộ thông tin thú cưng – điểm nâng cấp – tiến hoá trong cơ sở dữ liệu, vừa tăng tính minh bạch cho người dùng khi mỗi hành động đều có phản hồi rõ ràng. Nhờ vậy, hệ thống vừa duy trì được sự công bằng, vừa tạo trải nghiệm hứng thú trong quá trình nuôi dưỡng và phát triển thú cưng.

### 3.2.4 Sơ đồ hoạt động

**Quy trình học, ôn tập từ vựng và nhận thưởng**

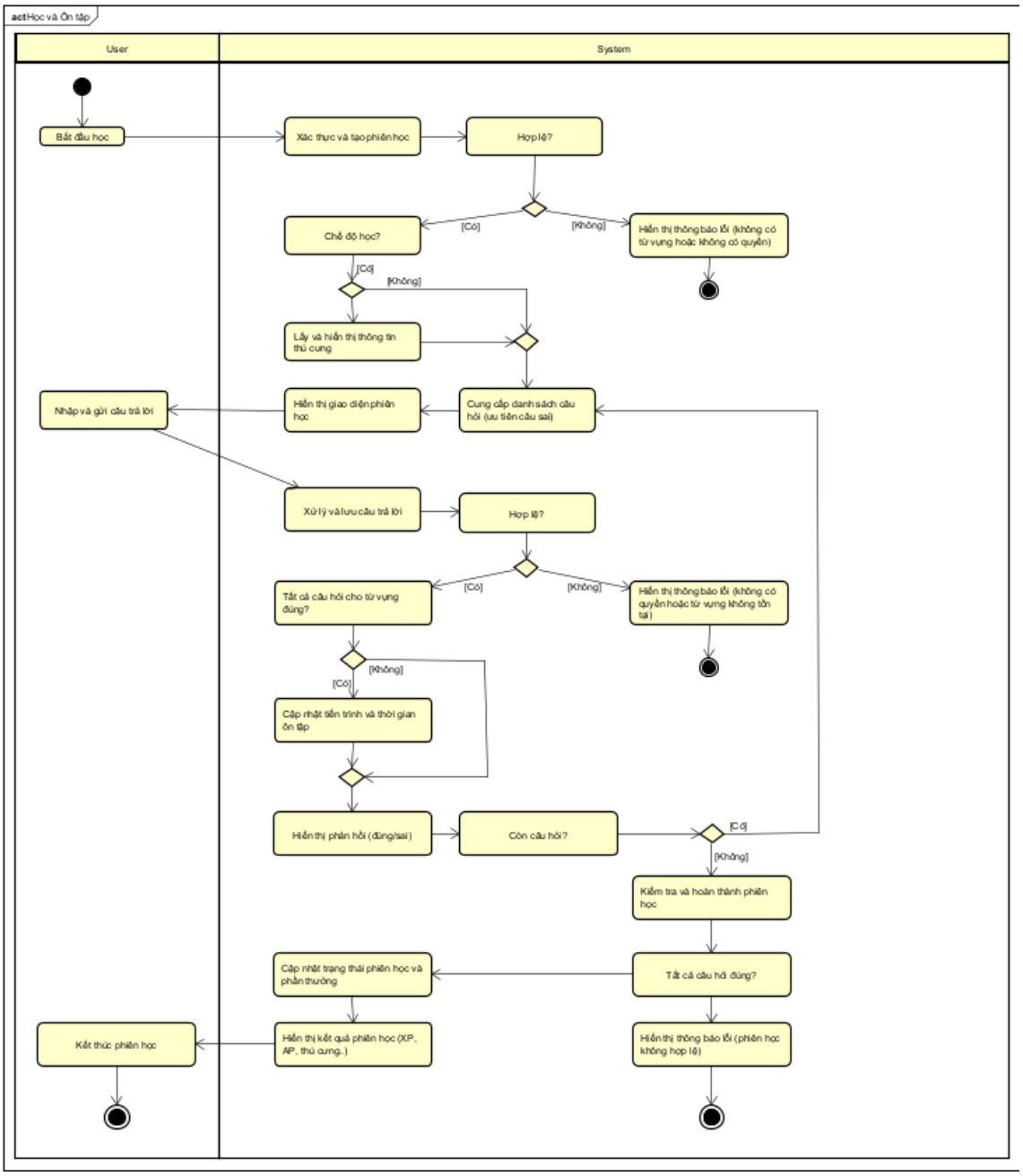


Figure 3.6: Sơ đồ hoạt động quy trình học, ôn tập từ vựng và nhận thưởng.

Sơ đồ mô tả luồng công việc chính khi người dùng bắt đầu một phiên học hoặc ôn tập:

- **Khởi động phiên học**: Người dùng yêu cầu bắt đầu học/ôn tập.
- **Xác thực và tạo phiên học**: Hệ thống xác thực thông tin và khởi tạo một phiên học trong cơ sở dữ liệu.
- **Điều kiện hợp lệ**:
  - Nếu hợp lệ → hiển thị giao diện học và (nếu có) thông tin thú cưng.
  - Nếu không hợp lệ → hiển thị thông báo lỗi và kết thúc.
- **Chuẩn bị nội dung**: Hệ thống truy vấn, sắp xếp danh sách câu hỏi (ưu tiên các câu sai trước, sau đó câu mới).
- **Quy trình học/ôn tập (vòng lặp)**:
  - Người dùng nhập câu trả lời.
  - Hệ thống kiểm tra, phản hồi ngay đúng/sai và lưu kết quả.
  - Nếu một từ vựng đã được trả lời đúng toàn bộ, tiến trình học và thời gian ôn tập được cập nhật.
- **Kết thúc vòng lặp**: Khi đã xử lý hết danh sách câu hỏi.
- **Hoàn tất phiên học**:
  - Nếu tất cả câu hỏi đều đúng → hệ thống cập nhật trạng thái phiên học, tính XP, phần thưởng và hiển thị kết quả (có animation bắt thú cưng nếu đủ điều kiện).
  - Nếu có lỗi hoặc dữ liệu không hợp lệ → hiển thị thông báo lỗi và kết thúc quy trình.

**Ý nghĩa:** Sơ đồ làm rõ các bước xử lý liên tiếp và các nhánh điều kiện trong toàn bộ phiên học. Khác với sơ đồ tuần tự vốn tập trung vào tương tác giữa đối tượng, sơ đồ hoạt động nhấn mạnh luồng xử lý công việc, sự lặp lại trong vòng trả lời–phản hồi, và điều kiện chuyển tiếp giữa các trạng thái. Điều này giúp minh họa cách quy trình học/ôn tập được triển khai liên tục và gắn liền với phần thưởng nhằm duy trì động lực học tập.

**Quy trình tạo bộ từ vựng**

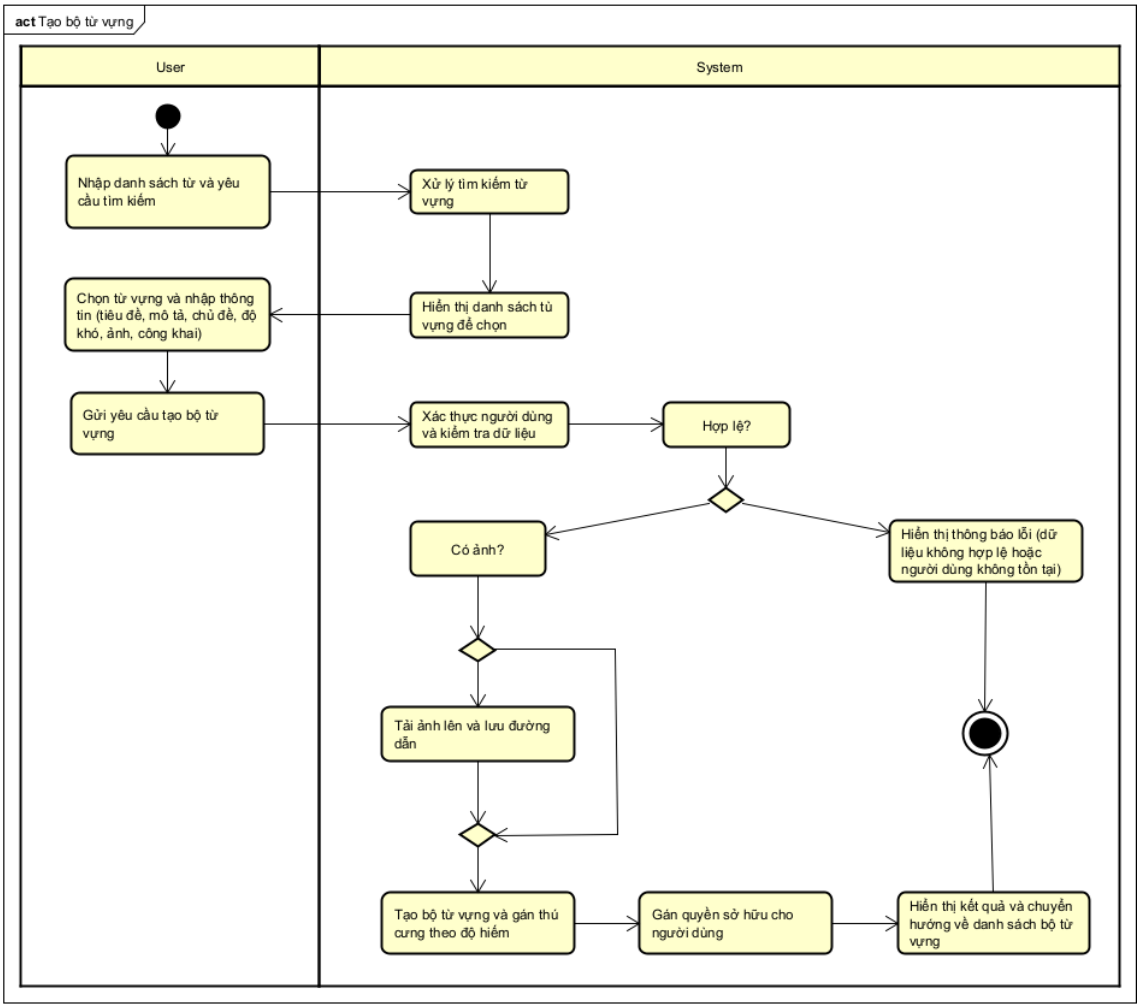


Figure 3.7: Sơ đồ hoạt động quy trình tạo bộ từ vựng.

Sơ đồ mô tả các bước chính khi người dùng tạo một bộ từ vựng mới:

- **Nhập dữ liệu ban đầu**: Người dùng nhập danh sách từ và gửi yêu cầu tìm kiếm.
- **Xử lý tìm kiếm**: Hệ thống truy vấn cơ sở dữ liệu để kiểm tra sự tồn tại của các từ và hiển thị kết quả tìm kiếm.
- **Lựa chọn và nhập thông tin**: Người dùng chọn các từ vựng cần thêm và điền thông tin chi tiết (tiêu đề, mô tả, chủ đề, độ khó, ảnh minh họa, chế độ công khai).
- **Xác thực và kiểm tra**: Hệ thống xác thực người dùng và kiểm tra dữ liệu đầu vào. nếu không hợp lệ → thông báo lỗi và kết thúc quy trình, nếu hợp lệ → tiếp tục.

- **Xử lý ảnh minh họa (nếu có)**: Nếu người dùng cung cấp ảnh, hệ thống tiến hành tải ảnh lên và lưu đường dẫn trong cơ sở dữ liệu.
- **Tạo bộ từ vựng**: Hệ thống tạo bộ từ mới, gán thú cưng tương ứng theo độ hiếm và phân quyền sở hữu cho người dùng.
- **Trả kết quả**: Hệ thống hiển thị kết quả tạo thành công và chuyển hướng về danh sách bộ từ cá nhân.

**Ý nghĩa:** Sơ đồ thể hiện cách hệ thống xử lý tuần tự và có điều kiện trong quá trình tạo bộ từ, đảm bảo dữ liệu đầu vào hợp lệ, phân quyền rõ ràng, đồng thời gắn kết với cơ chế thú cưng để tăng tính hấp dẫn. Điều này giúp quản lý chặt chẽ quyền sở hữu bộ từ và hỗ trợ khả năng mở rộng trong tương lai (ví dụ: chia sẻ công khai, trao đổi bộ từ giữa người dùng).

**Quy trình nâng cấp thú cưng**

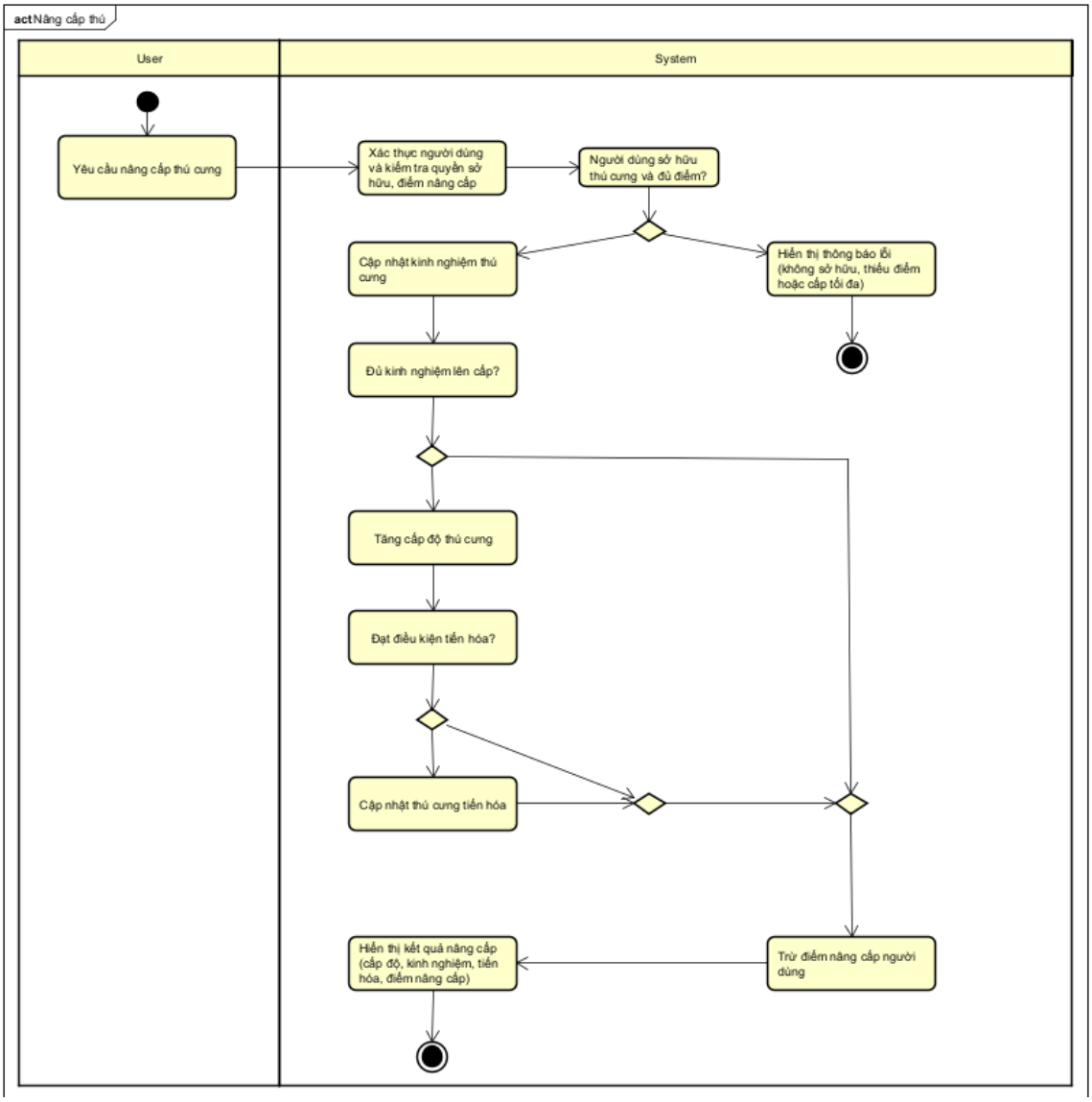


Figure 3.8: Sơ đồ hoạt động quy trình nâng cấp thú cưng.

Sơ đồ mô tả các bước xử lý chính khi người dùng thực hiện nâng cấp thú cưng:

- **Khởi động yêu cầu**: Người dùng gửi yêu cầu nâng cấp thú cưng.
- **Xác thực và kiểm tra**: Hệ thống xác thực người dùng, kiểm tra quyền sở hữu thú cưng và số điểm nâng cấp (AP), nếu điều kiện không hợp lệ (không sở hữu thú cưng, thiếu điểm AP, hoặc đã đạt cấp tối đa) → hệ thống hiển thị thông báo lỗi và kết thúc, Nếu hợp lệ → tiếp tục xử lý.

- **Cập nhật tiến trình**: Hệ thống cộng thêm kinh nghiệm cho thú cưng, nếu đạt ngưỡng → thú cưng được tăng cấp, Nếu đồng thời đạt điều kiện tiến hóa → hệ thống cập nhật trạng thái tiến hóa và ghi nhận thay đổi.
- **Hoàn tất nâng cấp**: Hệ thống trừ số điểm AP tương ứng của người dùng và lưu lại kết quả trong cơ sở dữ liệu.
- **Trả kết quả**: Hệ thống hiển thị thông tin cập nhật của thú cưng (cấp độ mới, kinh nghiệm, trạng thái tiến hóa nếu có, số AP còn lại).

**Ý nghĩa:** Quy trình đảm bảo trạng thái thú cưng được cập nhật chính xác và đồng bộ, đồng thời duy trì tính công bằng nhờ cơ chế kiểm tra điều kiện trước khi nâng cấp.

### 3.2.5 Sơ đồ trạng thái

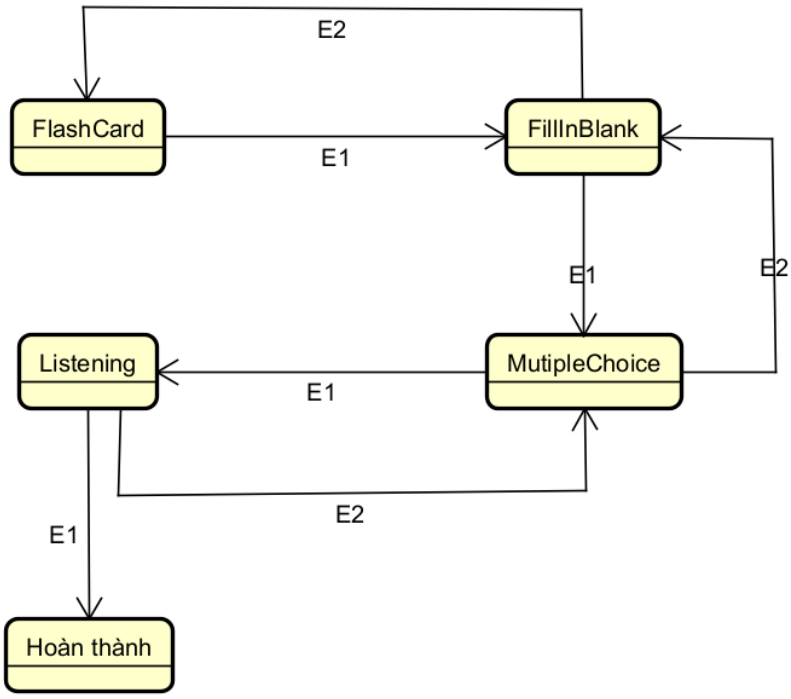


Figure 3.9: Sơ đồ trạng thái từ vựng trong phiên học.

Sơ đồ thể hiện các trạng thái mà một từ vựng phải trải qua trong suốt một phiên học, cùng với hai sự kiện chính chi phối quá trình chuyển trạng thái:

- Trạng thái 0 – FlashCard: Người học tiếp cận từ mới ở dạng thẻ nhớ.

- Trạng thái 1 – FillInBlank: Người học điền từ còn thiếu vào chỗ trống.
- Trạng thái 2 – MultipleChoice: Người học chọn đáp án đúng trong nhiều lựa chọn.
- Trạng thái 3 – Listening: Người học nghe và xác định từ vựng.
- Trạng thái 4 – Hoàn thành: Từ vựng được xem là đã học xong trong phiên hiện tại.

Các sự kiện chuyển trạng thái:

- E1 – Trả lời đúng: hệ thống tăng trạng thái, cho phép từ tiếp tục sang loại câu hỏi tiếp theo.
- E2 – Trả lời sai: hệ thống giảm trạng thái, đưa từ quay lại bước trước đó để ôn lại.

Nhờ cơ chế này, quá trình học được cá nhân hóa theo năng lực: từ vựng chỉ hoàn tất khi người học trả lời đúng liên tục qua tất cả các loại câu hỏi.

**Ý nghĩa**: Sơ đồ giúp minh họa vòng đời của một từ trong phiên học, cho thấy rõ cơ chế kiểm soát tiến độ dựa trên hiệu suất thực tế (đúng/sai), đảm bảo việc ghi nhớ chắc chắn và tránh tình trạng "học lướt".

### 3.2.6 Sơ đồ lớp

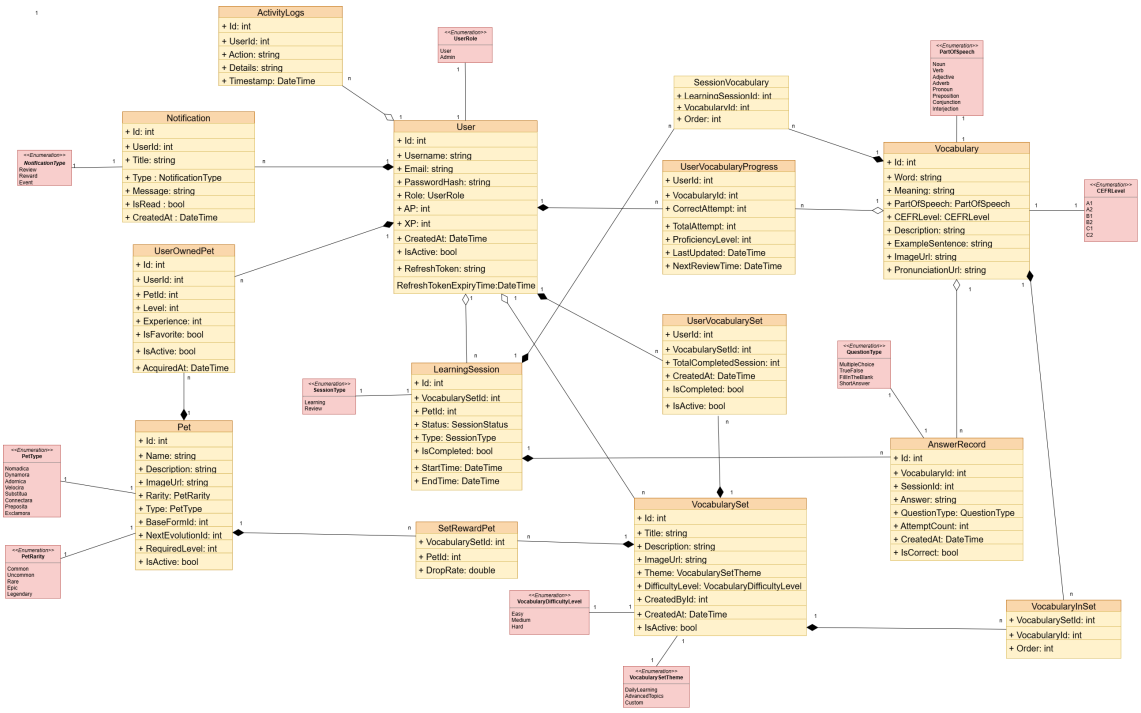


Figure 3.10: Sơ đồ lớp của hệ thống.

Sơ đồ mô tả cấu trúc hướng đối tượng của hệ thống, thể hiện các lớp nghiệp vụ và mối quan hệ giữa chúng. Đây là cầu nối từ phân tích yêu cầu sang thiết kế, làm cơ sở để sinh ra các bảng dữ liệu khi triển khai. Các lớp chính gồm:

- **User**: Đại diện cho người dùng hệ thống, lưu trữ thông tin định danh, bảo mật (mật khẩu băm), trạng thái tài khoản, điểm kinh nghiệm (XP), điểm nâng cấp thú cưng (AP) và các quan hệ đến tiến trình học, bộ từ, thú cưng, thông báo và log hoạt động.
- **VocabularySet**: Mô tả một bộ từ vựng theo chủ đề hoặc mục tiêu học, gồm tiêu đề, mô tả, chủ đề, độ khó, hình ảnh minh hoạ và trạng thái công khai. Mỗi bộ từ liên kết nhiều–nhiều với Vocabulary và có thể gắn thú cưng phần thưởng.
- **Vocabulary**: Đại diện cho một mục từ cụ thể với thông tin chi tiết: nghĩa, loại từ, mức CEFR, ví dụ, âm thanh và hình ảnh minh hoạ. Kết nối đến tiến trình học của người dùng, phiên học và kết quả trả lời.
- **LearningSession**: Quản lý một phiên học/ôn tập, lưu thời gian bắt đầu, kết thúc, loại phiên (học/ôn tập) và trạng thái hoàn thành. Phiên học chứa danh sách từ vựng được luyện và các bản ghi trả lời.
- **AnswerRecord**: Ghi lại thông tin trả lời từng câu hỏi trong một phiên học (nội dung trả lời, đúng/sai, số lần thử, loại câu hỏi, thời điểm).
- **UserVocabularyProgress**: Theo dõi trạng thái thành thạo của từng từ với từng người dùng, gồm số lần đúng, tổng số lần thử, cấp độ thành thạo và thời gian ôn tập tiếp theo.
- **Pet** và **UserOwnedPet**:
  - **Pet** mô tả loài thú ảo (tên, mô tả, độ hiếm, loại, hình thức tiến hoá, cấp yêu cầu).
  - **UserOwnedPet** quản lý việc người dùng sở hữu thú nào, cấp độ, kinh nghiệm, trạng thái yêu thích và kích hoạt.
- **Notification**: Gửi thông báo cho người dùng (nhắc nhở ôn tập, nhận thưởng, sự kiện).
- **ActivityLog**: Ghi nhận các hành động của người dùng phục vụ mục đích kiểm tra và phân tích hành vi.
- **Các lớp Enum**: Định nghĩa các giá trị hằng như vai trò người dùng, loại phiên học, độ hiếm thú cưng, loại câu hỏi, loại thông báo, độ khó bộ từ, v.v.

**Ý nghĩa:** Sơ đồ Class cung cấp cái nhìn tổng quan về kiến trúc hướng đối tượng, giúp lập trình viên hiểu rõ mối quan hệ giữa các thành phần và hỗ trợ ánh xạ trực tiếp sang mô hình dữ liệu quan hệ thông qua ORM (Entity Framework).

### 3.2.7 Lược đồ cơ sở dữ liệu vật lý

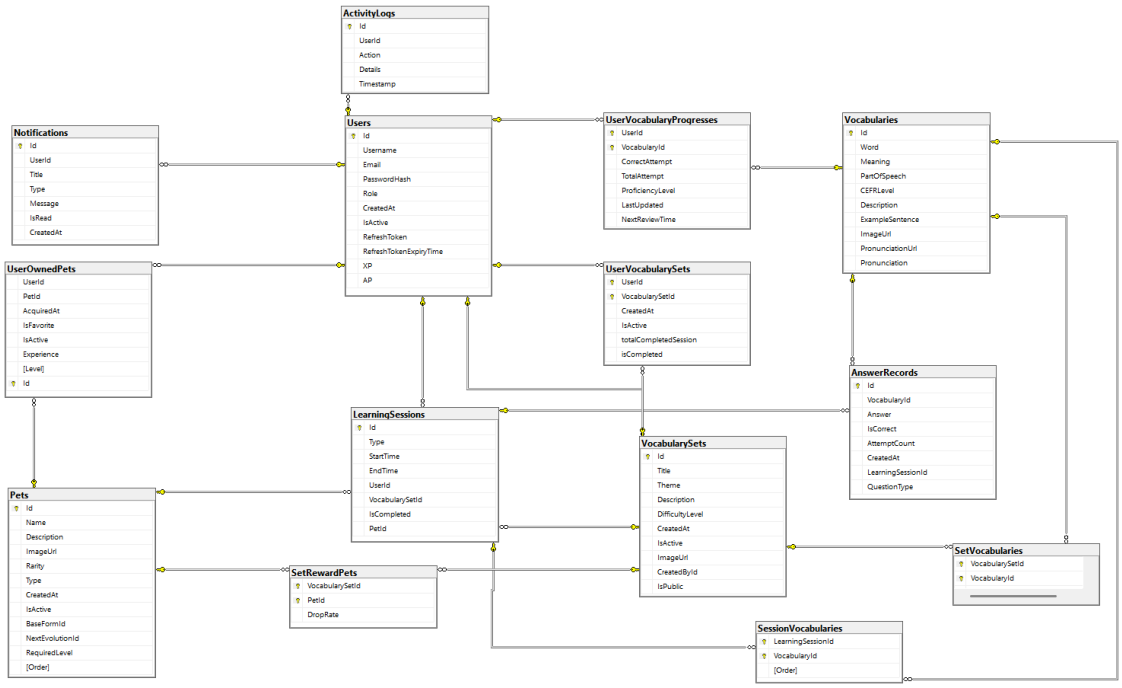


Figure 3.11: Lược đồ cơ sở dữ liệu vật lý.

Lược đồ mô tả cách các lớp trong sơ đồ Class được ánh xạ thành các bảng dữ liệu, kèm theo khoá chính (PK), khoá ngoại (FK) và quan hệ giữa chúng. Các bảng chính bao gồm:

- **Users**: Chứa thông tin định danh và trạng thái của người dùng. Quan hệ 1–N với LearningSessions, Notifications, ActivityLogs; quan hệ N–N với VocabularySets (UserVocabularySets), Vocabularies (UserVocabularyProgresses) và Pets (UserOwnedPets).
- **VocabularySets**: Lưu trữ các bộ từ vựng, có quan hệ N–N với Vocabularies (SetVocabularies), N–N với Pets (SetRewardPets) và 1–N với LearningSessions.
- **Vocabularies**: Lưu thông tin chi tiết từng từ, tham gia quan hệ N–N với VocabularySets và LearningSessions, 1–N với AnswerRecords.
- **LearningSessions**: Lưu thông tin từng phiên học/ôn tập, liên kết với Users, VocabularySets và SessionVocabularies, đồng thời chứa danh sách AnswerRecords.
- **AnswerRecords**: Ghi lại dữ liệu từng lần trả lời (đáp án, kết quả, loại câu hỏi, thời gian).
- **UserVocabularyProgresses**: Lưu trạng thái thành thạo từ vựng theo từng người dùng (tần suất đúng, cấp độ thành thạo, lịch ôn tập tiếp theo).
- **Pets** và **UserOwnedPets**: Lưu dữ liệu thú ảo và quan hệ sở hữu người dùng–thú, kèm thông tin cấp độ, kinh nghiệm và trạng thái.
- **SetRewardPets**: Xác định tỉ lệ rơi thú cưng khi hoàn thành một bộ từ.

- **Notifications**: Lưu thông báo gửi cho người dùng, trạng thái đã đọc.
- **ActivityLogs**: Lưu lịch sử hành động của người dùng để phục vụ kiểm tra, thống kê.
- **Các bảng liên kết khác**: SetVocabularies, SessionVocabularies, UserVocabularySets — đóng vai trò ánh xạ quan hệ nhiều–nhiều.

**Ý nghĩa:** Lược đồ cơ sở dữ liệu vật lý thể hiện cách dữ liệu được tổ chức trong SQL Server, bảo đảm toàn vẹn quan hệ, hỗ trợ truy vấn hiệu quả và mở rộng quy mô trong tương lai. Đây là nền tảng để hệ thống hoạt động ổn định và hỗ trợ các tính năng thống kê, gamification và cá nhân hóa tiến trình học.

## 3.3 Thiết kế giao diện

Thiết kế giao diện được thực hiện nhằm minh họa trực quan cách người dùng sẽ tương tác với hệ thống. Các giao diện được xây dựng bằng công cụ Mockflow, đảm bảo tính thống nhất về bố cục, màu sắc và trải nghiệm người dùng. Bộ giao diện tập trung vào các chức năng cốt lõi đã được xác định trong giai đoạn phân tích, đồng thời phản ánh rõ nét yếu tố gamification của hệ thống. Cụ thể, hệ thống bao gồm các nhóm giao diện chính sau:

### 3.3.1 Nhóm Dashboard và Người dùng

**Dashboard**: hiển thị thông tin tổng quan sau khi đăng nhập, gồm thông báo số lượng từ cần ôn tập, biểu đồ cột thể hiện phân bố từ vựng theo mức độ thành thạo, và thông tin cá nhân (avatar, tên, cấp độ, điểm số, số thú cưng sở hữu, streak).

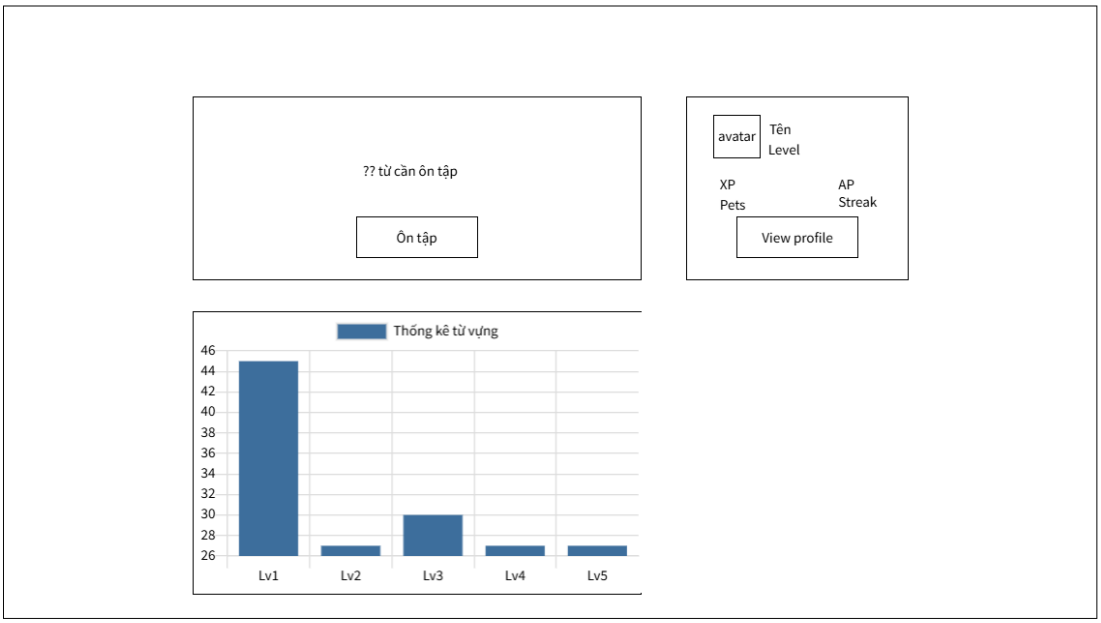


Figure 3.12: Hình thiết kế giao diện Dashboard

**Bảng xếp hạng**: liệt kê người dùng theo điểm số, gồm các cột thứ hạng, tên và điểm.

Figure 3.13: Hình thiết kế giao diện Bảng xếp hạng

### 3.3.2 Nhóm Bộ từ vựng

**Danh sách bộ từ**: hỗ trợ tìm kiếm và lọc theo trạng thái sở hữu hoặc loại bộ từ vựng; kèm chức năng tạo bộ từ mới.

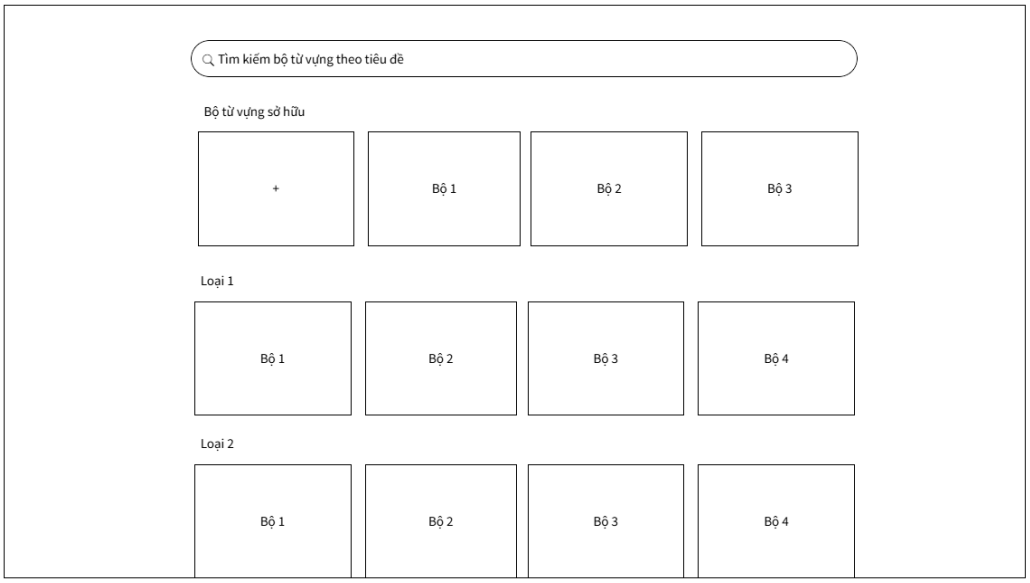


Figure 3.14: Hình thiết kế giao diện Danh sách bộ từ

**Tạo bộ từ**: gồm hai bước – (i) nhập danh sách từ khóa tìm kiếm; (ii) hiển thị các từ vựng gợi ý để người dùng lựa chọn, đồng thời nhập tiêu đề, mô tả, chủ đề, độ khó và tùy chọn công khai.

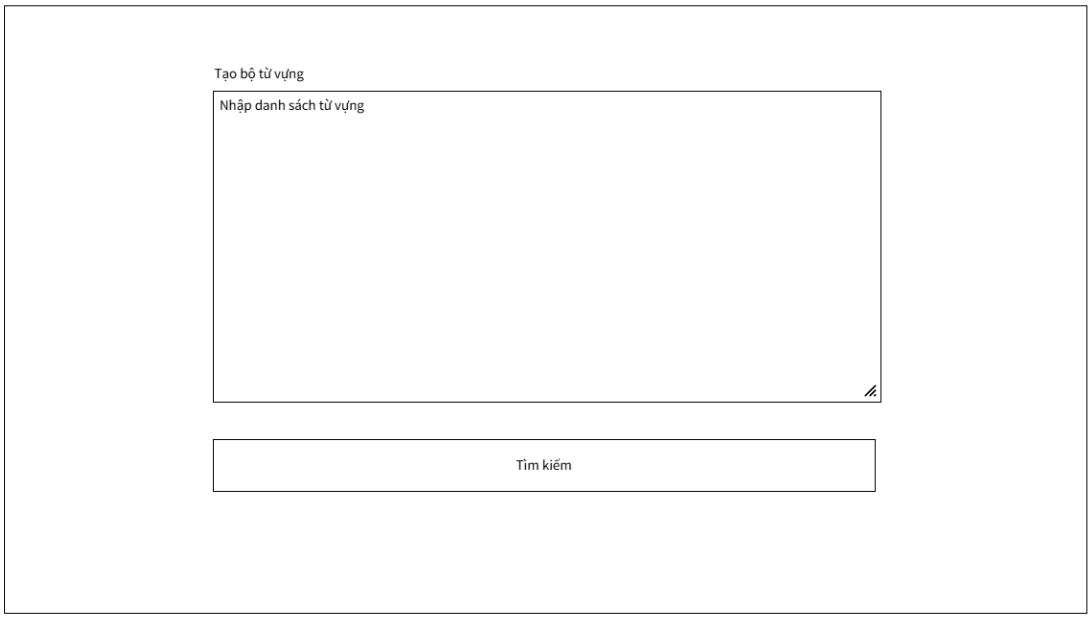


Figure 3.15: Hình thiết kế giao diện Tạo bộ từ (bước 1)

Figure 3.16: Hình thiết kế giao diện Tạo bộ từ (bước 2)

**Chi tiết bộ từ**: cho phép đăng ký sở hữu bộ từ để học, hiển thị danh sách từ vựng có phân trang, kèm tiến trình học của người dùng.

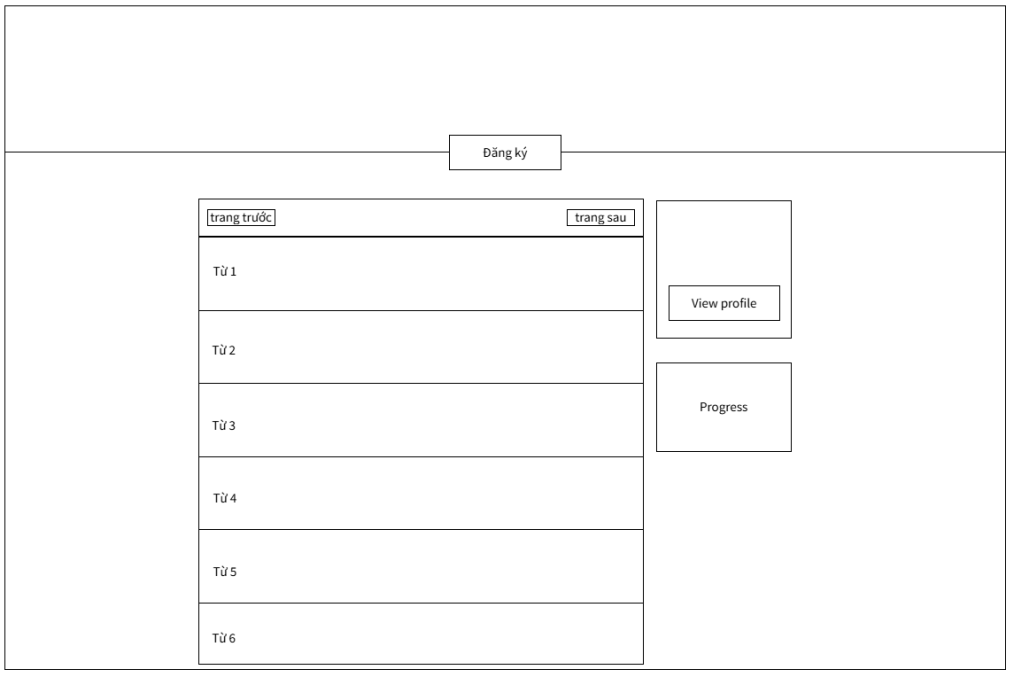


Figure 3.17: Hình thiết kế giao diện Chi tiết bộ từ

### 3.3.3 Nhóm Thú cưng (Pet)

**Danh sách thú cưng**: có thanh tìm kiếm theo tên, tùy chọn lọc theo độ hiếm hoặc loại thú cưng, và chế độ chỉ hiển thị thú cưng đã sở hữu.

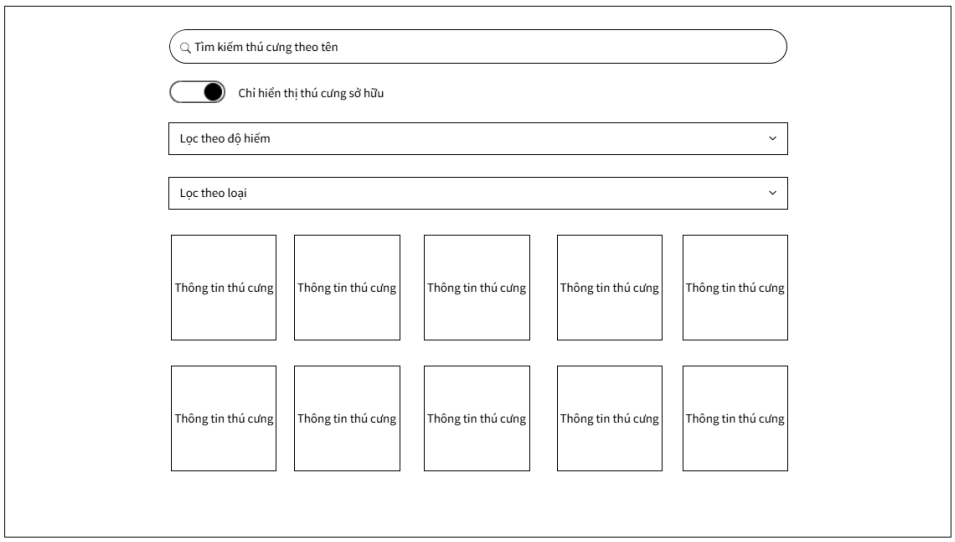


Figure 3.18: Hình thiết kế giao diện Danh sách thú cưng

**Chi tiết thú cưng**: hiển thị hình ảnh, thông tin chi tiết, thanh kinh nghiệm và cấp độ, kèm chức năng thăng cấp.

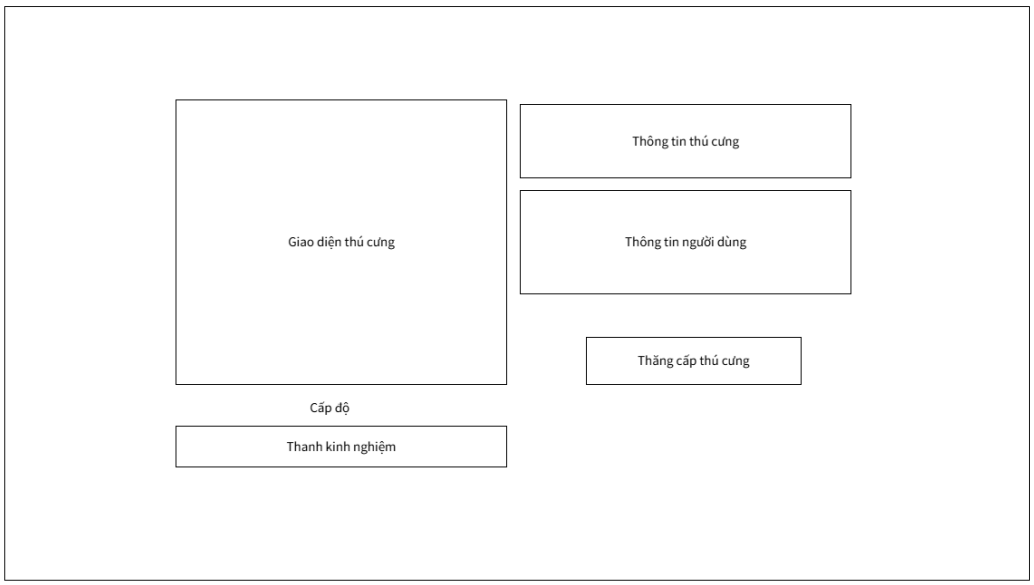


Figure 3.19: Hình thiết kế giao diện Chi tiết thú cưng

### 3.3.4 Nhóm Học và Ôn tập

**Phiên học từ vựng mới và ôn tập**: có bố cục chung gồm khung câu hỏi, hình ảnh thú cưng đồng hành, và khu vực trả lời. Hệ thống hỗ trợ bốn dạng câu hỏi: Flashcard: nút "đã xem"; Fill in the blank: ô nhập văn bản; Multiple choice: 4 lựa chọn dạng nút; Listening: ô nhập văn bản sau khi nghe âm thanh.

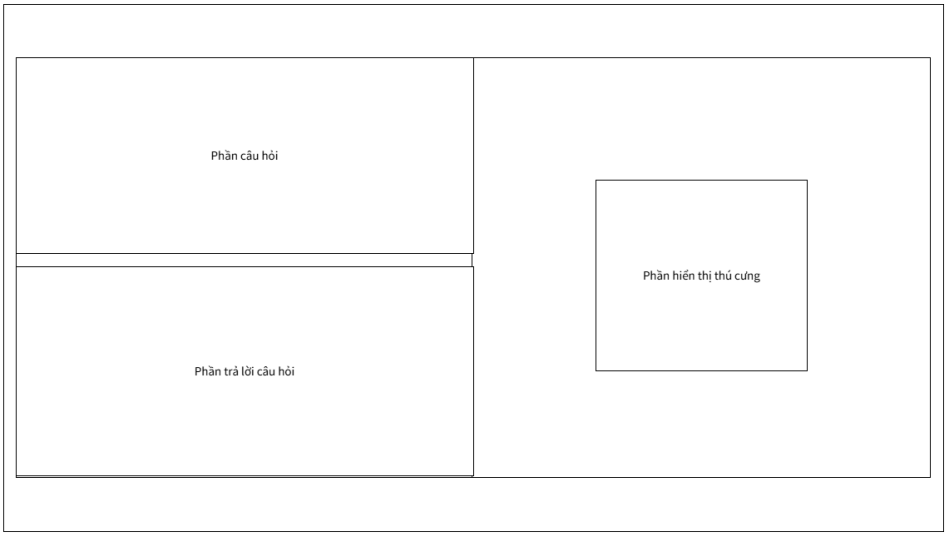


Figure 3.20: Hình thiết kế giao diện Học và Ôn tập

Nhìn chung, bộ giao diện đã bao quát hầu hết các trường hợp sử dụng quan trọng nhất của hệ thống, đảm bảo tính khả thi khi triển khai thực tế. Ngoài ra, bố cục và tính năng được thiết kế trực quan, nhằm tạo sự hứng thú và động lực cho người học trong suốt quá trình sử dụng hệ thống.

## 3.4 Kết quả đạt được

Sau giai đoạn phân tích, thiết kế và triển khai, hệ thống đã hiện thực đầy đủ các chức năng cốt lõi được đề ra trong mục tiêu đề tài. Sản phẩm không chỉ minh họa mô hình lý thuyết về học từ vựng kết hợp gamification mà còn là một hệ thống có thể sử dụng được ngay ở mức thử nghiệm. Các kết quả đạt được được trình bày theo từng nhóm chức năng dưới đây.

### 3.4.1 Phiên học và ôn tập từ vựng

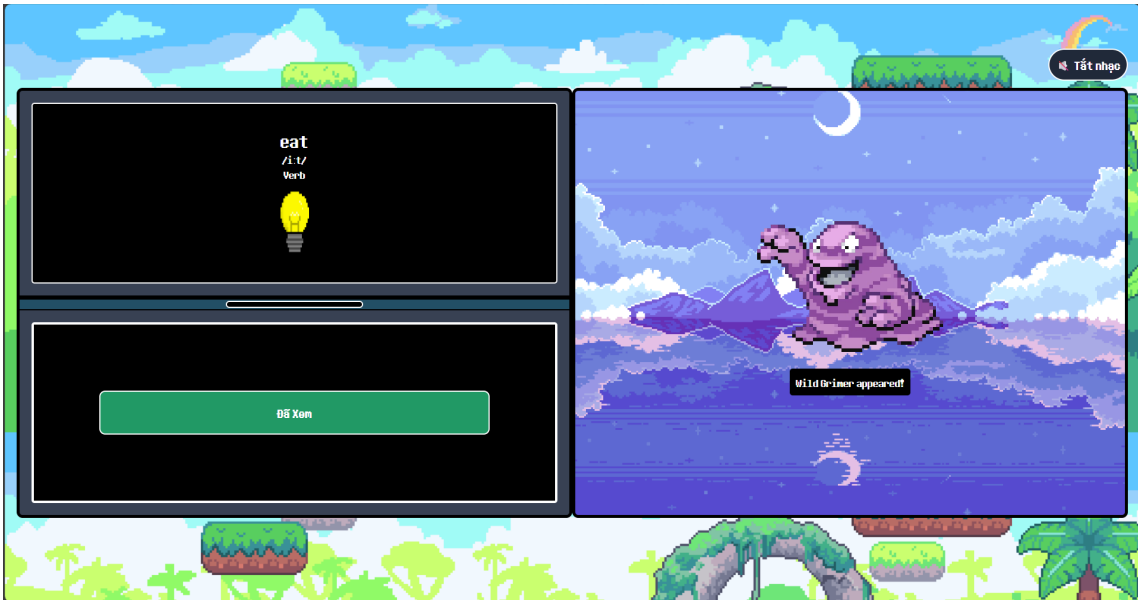


Figure 3.21: Màn hình thực hiện phiên học từ vựng.

Hệ thống hỗ trợ quy trình học theo mô hình **spaced repetition** và **active recall**, trong đó:

- Người dùng có thể chọn bộ từ vựng và khởi tạo phiên học hoặc ôn tập.
- Câu hỏi được lấy ra ưu tiên các từ từng trả lời sai, sau đó đến từ mới — giúp tập trung vào điểm yếu.
- Khi trả lời, kết quả được phản hồi ngay (đúng/sai) kèm hiệu ứng âm thanh và hình ảnh, tăng hứng thú học.
- Nếu tất cả câu hỏi của một từ vựng được trả lời đúng, tiến trình học của từ đó được nâng cấp (level up).
- Sau khi hoàn thành phiên học, hệ thống tính toán XP và trả thưởng (thú cưng, item) nếu đạt điều kiện.

Quy trình này đảm bảo nguyên tắc học chủ động và có phản hồi tức thì, vốn đã được chứng minh giúp cải thiện khả năng ghi nhớ dài hạn.

### 3.4.2 Tạo và quản lý bộ từ vựng

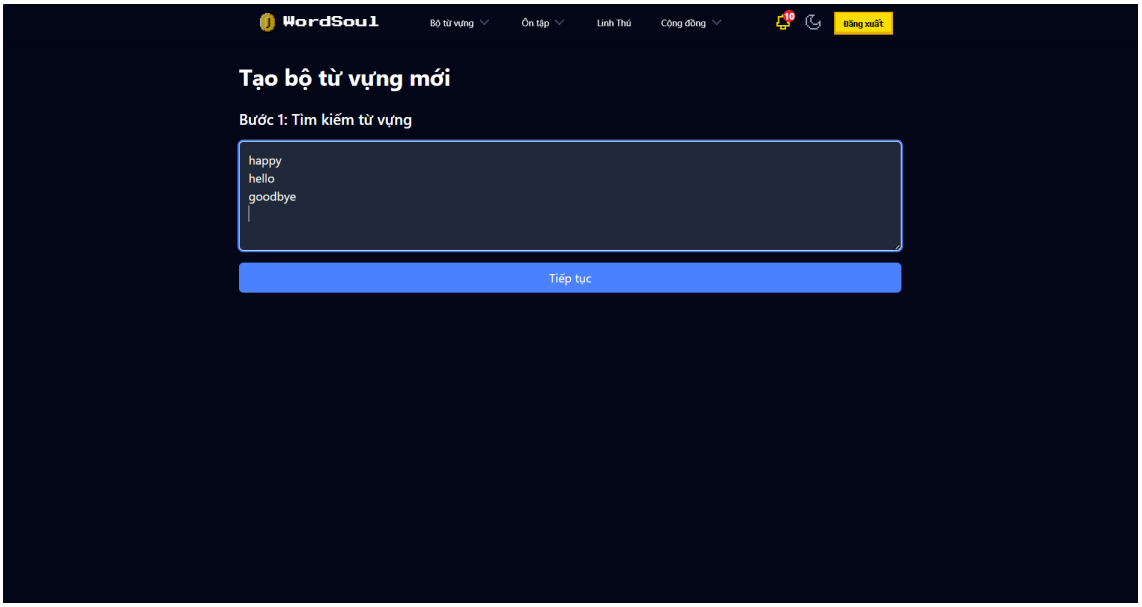


Figure 3.22: Màn hình tạo bộ từ vựng (Bước 1).

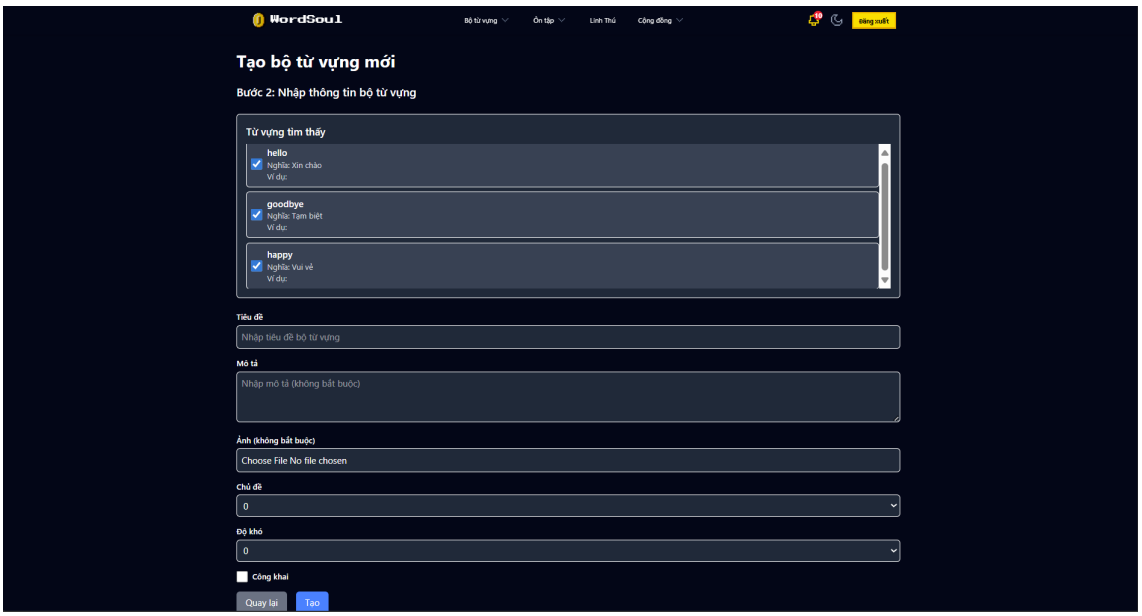


Figure 3.23: Màn hình tạo bộ từ vựng (Bước 2).

Chức năng này cho phép người dùng tự xây dựng nội dung học cá nhân:

- Nhập danh sách từ muốn tạo với bộ từ vựng để thực hiện tìm kiếm từ vựng trong hệ thống.
- Điền thông tin bổ sung như tiêu đề, mô tả, chủ đề, độ khó và tải ảnh minh họa.
- Quyết định chế độ công khai hay riêng tư để chia sẻ với người khác hoặc chỉ học cá nhân.
- Bộ từ vựng sau khi tạo được lưu trong cơ sở dữ liệu kèm quyền sở hữu, hỗ trợ chỉnh sửa về sau.

Nhờ đó, hệ thống hỗ trợ học tập có tính cá nhân hóa và mở rộng nội dung linh hoạt theo nhu cầu người học.

### 3.4.3 Hệ thống thú cưng và nâng cấp

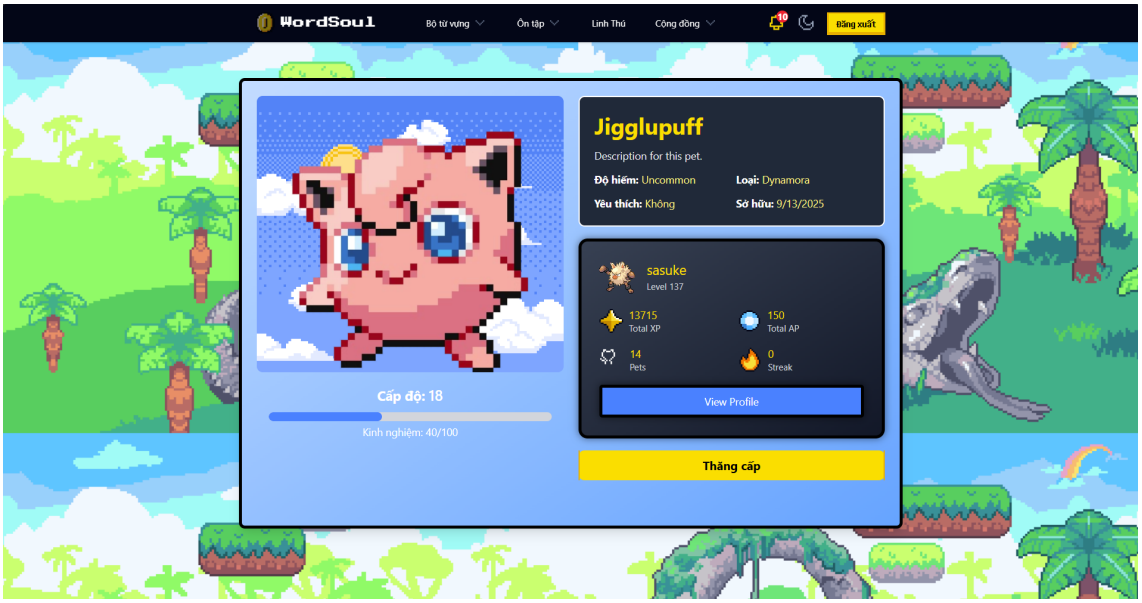


Figure 3.24: Màn hình nâng cấp thú cưng.

Hệ thống thú cưng là yếu tố gamification quan trọng:

- Người học có thể sở hữu nhiều thú cưng, mỗi thú có độ hiếm, cấp độ và khả năng tiến hóa.
- Trong quá trình học, người dùng tích lũy điểm AP; khi đủ có thể nâng cấp hoặc tiến hóa thú cưng.
- Hệ thống kiểm tra điều kiện (quyền sở hữu, AP, cấp tối đa), cập nhật dữ liệu và ghi nhật ký tiến hóa.

Chức năng này giúp tạo phần thưởng dài hạn, tăng động lực duy trì chuỗi học liên tục.

### 3.4.4 Dashboard người dùng

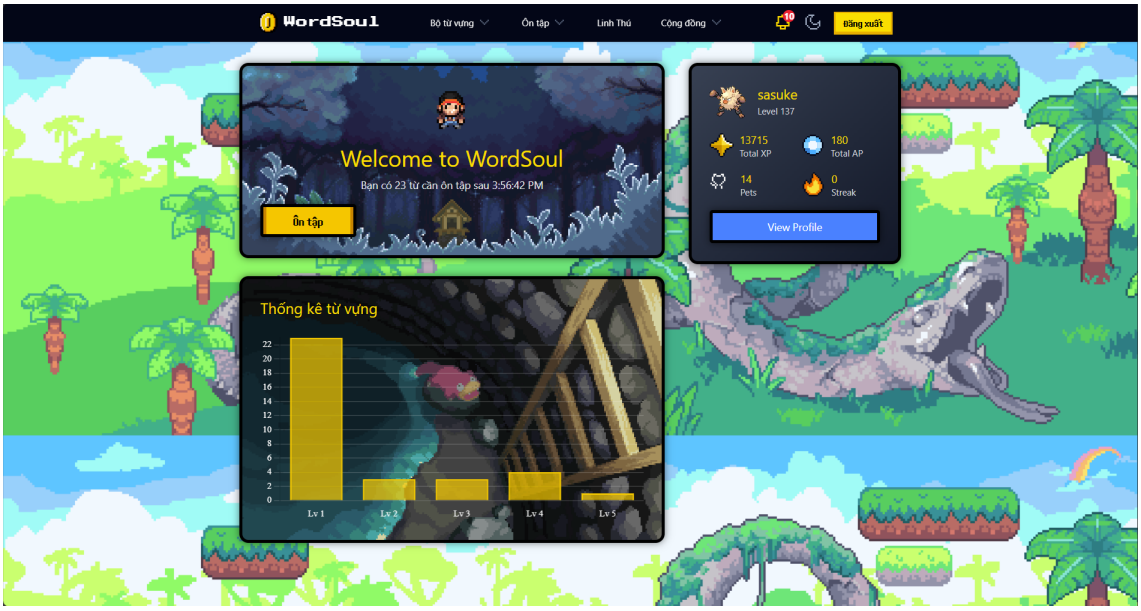


Figure 3.25: Màn hình Dashboard người dùng.

Màn hình Dashboard cung cấp góc nhìn tổng thể:

- Số lượng từ cần ôn tập trong ngày, biểu đồ phân bố mức độ thành thạo.
- Bảng thông tin người dùng: XP, AP, cấp độ, số thú cưng, chuỗi ngày học.
- Điều hướng nhanh đến các chức năng học, tạo bộ từ, hoặc quản lý thú cưng.

Đây là nơi trung tâm giúp người học theo dõi tiến độ và duy trì động lực.

### 3.4.5 Đánh giá chung

Hệ thống đã được kiểm thử với dữ liệu mẫu gồm 500 từ vựng, 5 bộ từ vựng và nhiều phiên học thử nghiệm. Các kết quả cho thấy:

- API phản hồi trong thời gian trung bình dưới 1.2 giây.
- Dữ liệu tiến trình được cập nhật chính xác, không phát sinh lỗi trong các tình huống nhập sai.
- Giao diện trực quan, quy trình học liền mạch, người dùng thử nghiệm đánh giá dễ thao tác.

Kết quả đạt được chứng minh tính khả thi của mô hình, đặt nền móng cho các giai đoạn phát triển tiếp theo như triển khai trên môi trường thực tế, bổ sung AI gợi ý lịch ôn tập hoặc phát triển ứng dụng di động.

# Chapter 4

# Kết luận và Hướng phát triển

## 4.1 Kết luận

Đề tài đã được triển khai với mục tiêu kết hợp các nguyên tắc khoa học về ghi nhớ (spaced repetition, retrieval practice) và yếu tố gamification (XP, phần thưởng, thú cưng) nhằm nâng cao hiệu quả học tập và duy trì động lực lâu dài cho người học.

Kết quả đạt được cho thấy hệ thống đã hiện thực thành công khoảng 80–90% các chức năng đã đặt ra trong phạm vi đề tài, bao gồm:

- Quản lý người dùng (đăng ký, đăng nhập, cập nhật thông tin cá nhân).
- Tạo và quản lý bộ từ vựng cá nhân, tìm kiếm và chọn từ để thêm vào bộ.
- Thực hiện các phiên học và ôn tập theo cơ chế *spaced repetition* và *active recall*, cung cấp phản hồi tức thì, cập nhật tiến trình học tự động.
- Tích hợp hệ thống gamification với điểm XP, AP, phần thưởng và thú cưng.
- Dashboard trực quan cho phép người dùng theo dõi tiến trình học, số lượng từ cần ôn tập, cấp độ cá nhân và tình trạng sưu tập thú cưng.

Bên cạnh các chức năng cốt lõi, hệ thống còn phát triển thêm nhiều chức năng hỗ trợ giúp nâng cao trải nghiệm người dùng, chẳng hạn như:

- Tìm kiếm và lọc bộ từ vựng, từ vựng và thú cưng theo nhiều tiêu chí (chủ đề, độ khó, độ hiếm).
- Bảng xếp hạng theo điểm kinh nghiệm (XP) để tạo động lực cạnh tranh lành mạnh giữa các người học.
- Quản lý thông báo nhắc nhở lịch ôn tập và thông báo nhận thưởng.

Về mặt kỹ thuật, hệ thống được thiết kế theo kiến trúc ba lớp, sử dụng ASP.NET Core Web API cho backend, React + TypeScript cho frontend và SQL Server làm cơ sở dữ liệu. Hệ thống đã được kiểm thử nội bộ, hoạt động ổn định và chứng minh được tính khả thi của mô hình đề xuất.

Đóng góp chính của đề tài là áp dụng các nguyên tắc học tập dựa trên bằng chứng khoa học vào một sản phẩm phần mềm cụ thể, đồng thời tích hợp gamification một cách hợp lý để duy

trì hứng thú học tập mà không tạo áp lực quá mức. Đây là một cách tiếp cận vừa mang tính học thuật vừa có tính ứng dụng thực tiễn, mở ra khả năng triển khai trong môi trường giáo dục trực tuyến.

## 4.2 Hạn chế

Do giới hạn thời gian và nguồn lực, hệ thống vẫn tồn tại một số hạn chế:

- Về tính năng: Chưa triển khai được ý tưởng *Pet Challenge* – cơ chế thách đấu thú cưng giữa các người dùng, nơi sát thương phụ thuộc vào tốc độ và độ chính xác trả lời.
- Về giao diện: Một số màn hình vẫn đang ở mức thử nghiệm, chưa tối ưu UX/UI và chưa hỗ trợ tốt cho thiết bị di động.
- Về học thuật: Hệ thống mới tập trung vào bài tập nhận biết (quiz, flashcard, nghe) mà chưa có tính năng luyện phát âm hoặc đánh giá *speaking*.
- Về đánh giá hiệu quả: Chưa có khảo sát thực nghiệm với người dùng thật để đo lường mức cải thiện khả năng ghi nhớ.

## 4.3 Hướng phát triển

Để hoàn thiện và mở rộng hệ thống trong tương lai, có thể triển khai các hướng sau:

- Hoàn thiện giao diện người dùng: Tối ưu trải nghiệm đa thiết bị, cải thiện hiệu ứng trực quan và hỗ trợ tùy chỉnh giao diện.
- Phát triển tính năng Pet Challenge: Xây dựng hệ thống chỉ số (HP, ATK, SPD) cho thú cưng, cơ chế chiến đấu dựa trên độ chính xác và tốc độ trả lời câu hỏi để tăng tính cạnh tranh.
- Bổ sung dạng bài tập nâng cao: Tích hợp API nhận diện giọng nói để chấm điểm phát âm, hỗ trợ luyện speaking.
- Mở rộng tính năng cộng đồng: Kết bạn, nhóm học chung, bảng xếp hạng nhiều tiêu chí (XP, streak, số lần thách đấu thành công).
- Đánh giá thực nghiệm: Tiến hành thử nghiệm với người dùng thật, thu thập phản hồi để tinh chỉnh thuật toán spaced repetition và gamification.
- Triển khai thực tế: Đưa hệ thống lên môi trường sản xuất, phục vụ nhiều người dùng đồng thời với khả năng mở rộng và bảo mật tốt.

Các định hướng trên không chỉ nhằm khắc phục các hạn chế hiện tại mà còn hướng đến việc biến hệ thống thành một nền tảng học từ vựng hiện đại, bền vững và thu hút người học lâu dài.

# Tài liệu tham khảo

## Tiếng Anh


[1] P. Nation. *Learning Vocabulary in Another Language*. Cambridge, UK: Cambridge University Press, 2001.

[2] N. J. Cepeda et al. "Spacing Effects in Learning: A Temporal Ridgeline of Optimal Retention". In: *Psychological Science* 19.11 (2008), pp. 1095–1102. DOI: 10.1111/j.1467-9280.2008.02209.x.

[3] J. D. Karpicke and H. L. Roediger. "The critical importance of retrieval for learning". In: *Science* 319.5865 (2008), pp. 966–968. DOI: 10.1126/science.1152408.

[4] Duolingo Team. *Duolingo 101: How to learn a language on Duolingo*. https://blog.duolingo.com/duolingo-101-how-to-learn-a-language-on-duolingo/. 2024.

[5] Quizlet. *Learn: Turn any flashcard set into adaptive practice*. https://quizlet.com/features/learn. 2024.

[6] Anki. *Anki — powerful, intelligent flashcards*. https://apps.ankiweb.net/. 2024.

[7] Memrise. *How does the spaced repetition system work?* https://memrise.zendesk.com/hc/en-us/articles/360015889057-How-does-the-spaced-repetition-system-work. 2024.

# Appendix A

# Mã nguồn một số phương thức quan trọng

Phụ lục này trình bày một số phương thức tiêu biểu trong mã nguồn của hệ thống, được lựa chọn dựa trên mức độ quan trọng trong việc minh họa kiến trúc hệ thống và giải quyết các yêu cầu nghiệp vụ. Các API này không chỉ thực hiện các thao tác CRUD cơ bản mà còn bao hàm các thuật toán xử lý dữ liệu, cơ chế học tập theo *spaced repetition*, cũng như yếu tố *gamification* thông qua hệ thống thú cưng.

Mục tiêu của phụ lục là cung cấp cho người đọc cái nhìn rõ ràng về cách hiện thực hoá các yêu cầu chức năng, thay vì liệt kê toàn bộ mã nguồn.

## A.1 API Quản lý Phiên Học

**Mục đích:** Khởi tạo, quản lý và hoàn tất các phiên học hoặc ôn tập của người dùng. Đây là điểm vào (entry point) chính để bắt đầu luồng học.

Listing A.1: Tạo một phiên học dựa trên bộ từ vựng đã chọn

```
[Authorize(Roles = "User")]
[HttpPost("{vocaSetId}")]
public async Task<IActionResult> CreateLearningSession(int vocaSetId)
{
    var userId = User.GetUserId();
    if (userId == 0) return Unauthorized();
    if (vocaSetId <= 0) return BadRequest("Invalid VocabularySet ID");

    try
    {
        var learningSessionDto = await _learningSessionService.CreateLearningSessionA
        return Ok(learningSessionDto);
    }
    catch (InvalidOperationException ex)
    {
        _logger.LogWarning(ex, "No unlearned vocabularies found.");
        return NotFound(ex.Message);
```

```
    }
}
```

API trên thể hiện rõ cách hệ thống:

- Xác thực người dùng thông qua JWT và Role.
- Kiểm tra quyền sở hữu bộ từ vựng.
- Xử lý tình huống không còn từ để học và trả về mã lỗi phù hợp.

## A.2 Thuật Toán Cập Nhật Tiến Trình Học

**Mục đích:** Lưu kết quả trả lời và áp dụng cơ chế *spaced repetition* để xác định thời điểm ôn tập tiếp theo.

Listing A.2: Cập nhật tiến trình học và tính ngày ôn tập

```
progress.ProficiencyLevel = Math.Min(5,
    1 + (int)Math.Floor(Math.Log(Math.Max(1, progress.CorrectAttempt), 1.5)));

int daysToAdd = progress.ProficiencyLevel switch
{
    1 => 1,
    2 => 2,
    3 => 4,
    4 => 16,
    _ => 256
};

progress.NextReviewTime = DateTime.UtcNow.AddDays(daysToAdd);
```

Đây là phần mang tính học thuật quan trọng, vì nó giúp hệ thống tối ưu tần suất xuất hiện của từ vựng, đảm bảo người học ghi nhớ lâu hơn mà không quá tải.

## A.3 API Tạo Bộ Từ Vựng

**Mục đích:** Tạo bộ từ vựng mới và tự động gán thú cưng phần thưởng theo độ hiếm.

Listing A.3: Tự động gán Pet thưởng cho bộ từ vựng

```
var petDistribution = new Dictionary<PetRarity, (int Count, double DropRate)>
{
    { PetRarity.Common, (10, 0.4) },
    { PetRarity.Uncommon, (5, 0.25) },
    { PetRarity.Rare, (3, 0.15) },
    { PetRarity.Epic, (2, 0.05) },
    { PetRarity.Legendary, (1, 0.01) }
};

foreach (var (rarity, (count, dropRate)) in petDistribution)
```

```
{
    var pets = await _petRepository.GetRandomPetsByRarityAsync(rarity, count);
    vocabularySet.SetRewardPets.AddRange(pets.Select(pet => new SetRewardPet
    {
        PetId = pet.Id,
        DropRate = dropRate
    }));
}
```

Đoạn mã này cho thấy cách hệ thống sử dụng logic xác suất để phân phối phần thưởng, góp phần tăng tính hấp dẫn của quá trình học.

## A.4 API Nâng Cấp Thú Cưng

**Mục đích:** Xử lý logic tăng kinh nghiệm, tăng cấp độ và tiến hoá thú cưng, từ đó gắn kết yếu tố trò chơi vào hệ thống học.

Listing A.4: Nâng cấp và tiến hóa thú cưng

```
while (userOwnedPet.Experience >= 100)
{
    userOwnedPet.Level++;
    userOwnedPet.Experience -= 100;
    isLevelUp = true;

    if (currentPet.RequiredLevel != null &&
        userOwnedPet.Level >= currentPet.RequiredLevel &&
        currentPet.NextEvolutionId.HasValue)
    {
        var evolvedPet = await _petRepository.GetPetByIdAsync(currentPet.NextEvolutio
        if (evolvedPet != null)
        {
            userOwnedPet.PetId = evolvedPet.Id;
            isEvolve = true;
            break;
        }
    }
}
```

API này minh họa cách hệ thống quản lý trạng thái phức tạp của thú cưng, đồng thời đảm bảo tính nhất quán dữ liệu khi người dùng nâng cấp nhiều lần liên tiếp.

## A.5 Nhận Xét Chung

Các API trên thể hiện:

- Cách hệ thống kết hợp giữa **xác thực – xử lý nghiệp vụ – lưu trữ dữ liệu**.
- Việc áp dụng thuật toán học tập và gamification.

- Thiết kế chuẩn RESTful, tách biệt Controller – Service – Repository, dễ mở rộng trong tương lai.

Các API CRUD cơ bản khác (xem/xoá/cập nhật Pet, tìm kiếm bộ từ vựng, quản lý người dùng) không được liệt kê ở đây để tránh trùng lặp và giữ tính cô đọng cho phụ lục.

# Appendix B

# Một Số Màn Hình Bổ Sung

Phụ lục này giới thiệu thêm một số màn hình giao diện của hệ thống , minh hoạ các chức năng phụ trợ như lọc, tìm kiếm, xem chi tiết và điều hướng học tập. Các màn hình này không được mô tả chi tiết trong chương Kết quả đạt được nhưng đóng vai trò quan trọng trong việc nâng cao trải nghiệm người dùng.

## B.1 Màn hình Danh sách Thú cưng

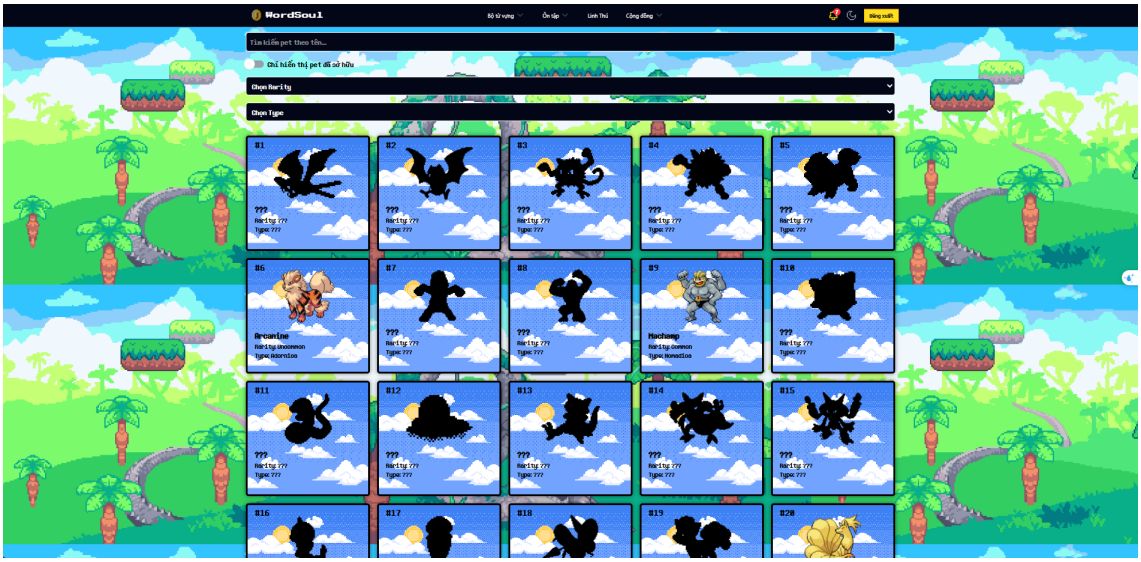


Figure B.1: Màn hình danh sách thú cưng với bộ lọc nâng cao.

Màn hình này hiển thị toàn bộ thú cưng trong hệ thống và hỗ trợ:

- Lọc theo tên thú cưng hoặc chỉ hiển thị thú đã sở hữu.
- Lọc theo độ hiếm (*Common, Rare, Epic, Legendary*) và loại (*Beast, Spirit, Machine,...*).

- Hiển thị trực quan tình trạng sở hữu, cấp độ hiện tại và tiến trình tiến hoá.

Chức năng này giúp người học dễ dàng theo dõi bộ sưu tập, tăng động lực thu thập đầy đủ thú cưng.

## B.2 Màn hình Chi tiết Bộ từ vựng

Figure B.2: Màn hình chi tiết bộ từ vựng.

Màn hình này cung cấp thông tin toàn diện về một bộ từ:

- Hiển thị tiêu đề, mô tả, ảnh minh họa, chủ đề, độ khó và thú cưng thưởng.
- Nút **Đăng ký bộ từ vựng** (nếu người dùng chưa sở hữu) và nút **Học** (nếu đã sở hữu).
- Nút **Đã hoàn thành** được disable khi người dùng đã học xong toàn bộ bộ từ.
- Danh sách tất cả các từ trong bộ, kèm tiến trình (ví dụ: số lần đúng, mức độ thành thạo).

Màn hình này là trung tâm định hướng học tập, giúp người dùng quyết định đăng ký hoặc tiếp tục học.

## B.3 Màn hình Danh sách Bộ từ vựng

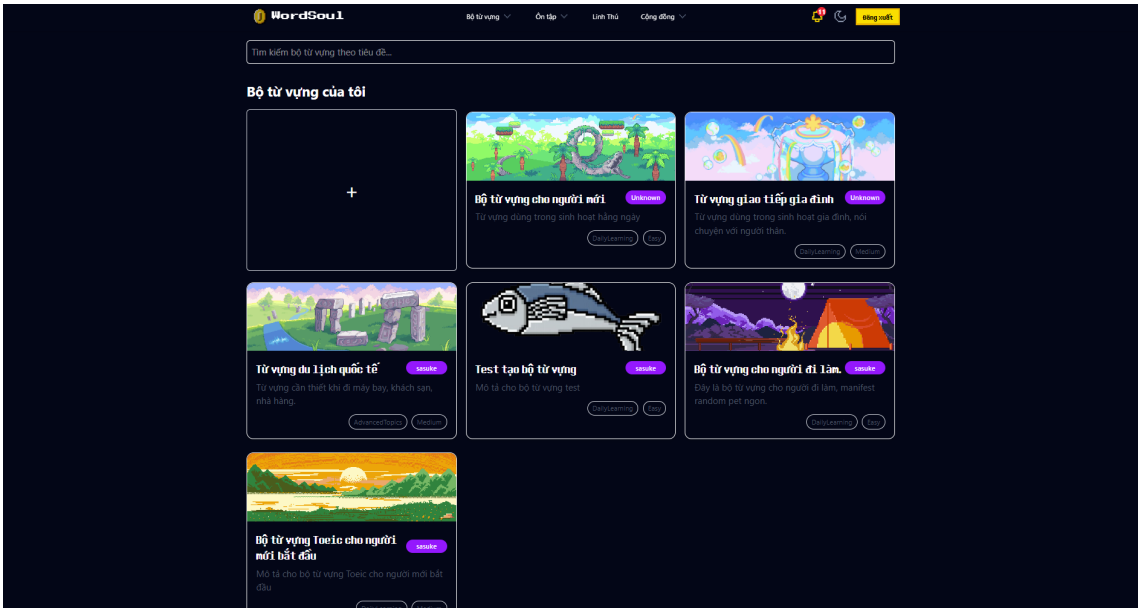


Figure B.3: Màn hình danh sách bộ từ vựng.

Màn hình này cho phép duyệt và tìm kiếm bộ từ vựng:

- Lọc theo **chủ đề** (Theme) và mức độ khó.
- Tìm kiếm theo từ khóa để nhanh chóng tìm bộ từ liên quan.
- Hiển thị trạng thái sở hữu và số lượng từ còn phải học trong từng bộ.

## B.4 Ý nghĩa và Vai trò

Các màn hình trong phụ lục này thể hiện sự quan tâm đến **trải nghiệm người dùng (UX)**:

- Tối ưu quá trình điều hướng: người học dễ tìm bộ từ, chọn pet hoặc kiểm tra tiến trình.
- Tăng tính cá nhân hóa: hiển thị rõ tình trạng sở hữu, tiến trình học, và khuyến khích hoàn thành.
- Hỗ trợ việc mở rộng sau này: thêm bộ lọc nâng cao, sắp xếp theo mức độ phổ biến hoặc ngày tạo.

Những màn hình này đóng vai trò quan trọng trong việc biến hệ thống thành một nền tảng học tập có tính cộng đồng và khuyến khích người học quay lại hằng ngày.